

A Multi-Stage Deep Learning Framework for Smartphone E-Waste Classification, Damage Assessment, Metal Value Estimation, and Recycling Recommendation

by

Ahanaf Mokammel Omi

21301504

Parijat Debnath Ratul

21101280

Abdullah Ibna Siddiquie

21301591

Tasnova Rahman

24341260

Anika Toushin Sinthia

24341251

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
June 2026.

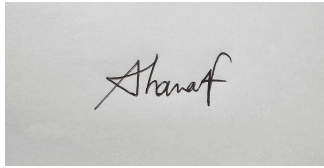
© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is our own original work while completing the degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:



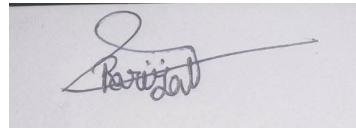
Ahanaf Mokammel Omi

21301504



Abdullah Ibna Siddique

21301591



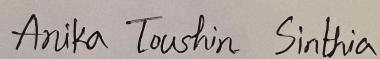
Parijat Debnath Ratul

21101280



Tasnova Rahman

24341260



Anika Toushin Sinthia

24341251

Approval

The thesis titled “*A Multi-Stage Deep Learning Framework for Smartphone E-Waste Classification, Damage Assessment, Metal Value Estimation, and Recycling Recommendation*” submitted by

1. Ahanaf Mokammel Omi (21301504)
2. Parijat Debnath Ratul (21101280)
3. Abdullah Ibna Siddiquie (21301591)
4. Tasnova Rahman (24341260)
5. Anika Toushin Sinthia (24341251)

of Summer, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science and Engineering in Summer 2026.

Examining Committee:

Supervisor:
(Member)



MD. Khalilur Rahman

Professor
Department of Computer Science and Engineering
Brac University

Co-Supervisor:
(Member)



Riazul Islam Rifat

lecturer
Department of Computer Science and Engineering
Brac University

Thesis Coordinator:
(Member)

Name of Thesis Coordinator

Designation
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Chairperson
Department of Computer Science and Engineering
Brac University

Ethics Statement

This research involved the collection, curation, and use of smartphone images for training and evaluating machine learning models. All images were collected from publicly available online sources. No personally identifiable information was collected or retained. The dataset does not include images of individuals and is used solely for academic research purposes. No human subjects were involved in data collection, and no approval from an ethics board was required.

Abstract

Smartphone e-waste is one of the fastest-growing and most material-rich waste streams globally. Proper handling requires identifying the device model, assessing its physical condition, evaluating damage severity, estimating recoverable material value, and recommending a handling pathway. This thesis presents a five-stage deep learning pipeline that automates these tasks from a single smartphone image.

Stage-A classifies the smartphone model (iphone_8, iphone_x, iphone_11, iphone_13) using DINOv2 ViT-B/14, achieving **91.91%** test accuracy. Stage-B classifies device condition (normal or broken) using Swin Transformer V2 Tiny, achieving **92.12%** test accuracy (best validation accuracy **95.44%**). Stage-C classifies damage severity (minor or major damage) using MaxViT-Tiny, achieving **94.67%** test accuracy. Stage-C executes only when Stage-B predicts broken. Stage-D estimates recyclable metal value in Bangladeshi Taka (BDT) using a metal composition database and a hybrid pricing system that calls MetalpriceAPI for live or delayed market prices with a fallback CSV for unavailable metals or API failure. Stage-E generates a handling recommendation (resell as-is, repair then resell, recycle, or manual review) from a deterministic rule engine.

Three model families are compared across all three classification stages: CNN baselines (DenseNet121), YOLO11x-cls, and Vision Transformer-based models. ViT-based models outperform both baselines across all stages, with improvements of up to 15.78 percentage points over CNN baselines. The complete pipeline is integrated into a FastAPI/React web demonstration system.

Keywords: Smartphone e-waste, deep learning, Vision Transformer, DINOv2, Swin Transformer V2, MaxViT, image classification, metal value estimation, MetalpriceAPI, decision support, recycling recommendation.

Dedication

To our families, for their unwavering patience and support throughout this journey.

To our supervisor, MD. Khalilur Rahman, for his guidance and trust.

*And to every recycler and environmental worker who handles e-waste daily — this
work is for you.*

Acknowledgement

We would like to express our sincere gratitude to our supervisor, MD. Khalilur Rahman, Professor, Department of Computer Science and Engineering, Brac University, for his continuous guidance, encouragement, and constructive feedback throughout the duration of this thesis.

We are grateful to the Department of Computer Science and Engineering at Brac University for providing the academic environment and resources that made this work possible. We thank the Ayesha Abed Library for providing access to research databases and digital resources.

We acknowledge the open-source communities behind PyTorch, HuggingFace Transformers, timm, Ultralytics YOLO, FastAPI, and React, whose tools formed the technical foundation of this project.

Finally, we thank our families and friends for their patience, motivation, and moral support during the long hours this project required.

Table of Contents

Declaration	i
Approval	ii
Ethics Statement	iv
Abstract	v
Dedication	vi
Acknowledgment	vii
Table of Contents	viii
List of Figures	xii
List of Tables	xiii
Nomenclature	xiv
1 Introduction	1
1.1 Background of Smartphone E-Waste	1
1.2 Problem Statement	2
1.3 Motivation	2
1.4 Research Objectives	3
1.5 Research Questions	3
1.6 Scope of the Study	4
1.7 Expected Contributions	4
1.8 Thesis Organisation	5
2 Literature Review	6
2.1 E-Waste and Smartphone Recycling	6
2.2 Computer Vision in Waste Management	7
2.3 CNN-Based Image Classification	7

2.4	YOLO Classification Models	8
2.5	Vision Transformers	8
2.6	DINOv2, Swin Transformer V2, and MaxViT	9
2.7	MetalpriceAPI and Metal Value Estimation	9
2.8	Evaluation, Reproducibility, and Dataset Documentation	10
2.9	Research Gap	10
3	Methodology	11
3.1	Full Five-Stage System Overview	11
3.2	Custom Dataset Construction Overview	12
3.3	Dataset Sources	12
3.3.1	Online Image Collection	12
3.3.2	Real-World Data Collection from Repair and Scrap Shops . .	13
3.4	Stage-Wise Dataset Design	13
3.4.1	Stage-A Dataset: Smartphone Model Classification	13
3.4.2	Stage-B Dataset: Normal vs. Broken Condition Classification .	14
3.4.3	Stage-C Dataset: Minor vs. Major Damage Severity Classifi- cation	14
3.5	Class Definitions and Labelling Rules	14
3.5.1	Smartphone Model Label Criteria	14
3.5.2	Normal and Broken Class Criteria	15
3.5.3	Minor and Major Damage Class Criteria	15
3.5.4	Borderline Case Handling	15
3.6	Data Cleaning and Preprocessing	15
3.7	Data Augmentation Strategy	16
3.8	Dataset Splitting Strategy	16
3.9	Model Families Tested	16
3.10	Transfer Learning and Fine-Tuning	16
3.11	Training Configuration	17
3.12	Evaluation Metrics	17
3.13	Model Selection Criteria	18
3.14	Stage-D Metal Value Estimation Methodology	18
3.14.1	Metal Composition Database	18
3.14.2	MetalpriceAPI Integration	18
3.14.3	USD-to-BDT Conversion	19
3.14.4	Fallback Price Mechanism	19
3.14.5	Limitations of Estimated Recyclable Value	19
3.15	Stage-E Recommendation Methodology	19
3.16	Reproducibility	21

3.17	Threats to Validity	21
3.18	Dataset Limitations	21
4	Implementation	22
4.1	Project Architecture Overview	22
4.2	Final ViT Pipeline Script: <code>run_vit_pipeline.py</code>	22
4.2.1	Stage-B Conditional Logic for Stage-C	23
4.2.2	MetalpriceAPI Integration in Stage-D	23
4.2.3	Output Structure	24
4.3	FastAPI Web API	24
4.4	React Frontend Interface	24
4.5	Training Scripts	24
4.6	Stage-D Database Files	24
4.7	Pipeline Output: MetalpriceAPI Enabled Run	26
4.8	Representative Pipeline JSON Outputs	26
5	Results and Analysis	28
5.1	Dataset Statistics	28
5.2	CNN Baseline Results	28
5.3	YOLO11x-cls Comparison Results	29
5.4	Vision Transformer Comparison Results	29
5.5	Cross-Family Model Comparison	29
5.6	Stage-A Detailed Result Analysis	30
5.7	Stage-B Detailed Result Analysis	32
5.8	Stage-C Detailed Result Analysis	34
5.9	Final Pipeline Summary	36
5.10	Stage-D Live Pricing Validation	36
5.11	Estimated Cascading Pipeline Accuracy	37
5.12	Estimated Cascading Pipeline Accuracy	37
5.13	Normal Phone Pipeline Output Example	38
5.14	Broken Phone Pipeline Output Example	40
5.15	Training Curve Analysis	40
5.16	Why ViT Was Selected for Final Deployment	41
6	Discussion	42
6.1	Overall Pipeline Performance	42
6.2	Stage-A Analysis: Model Classification	42
6.3	Stage-B Analysis: Condition Classification	43
6.4	Stage-C Analysis: Damage Severity	43
6.5	Cross-Family Comparison Discussion	44

6.6	Stage-D Metal Value Estimation	44
6.7	Limitations	44
6.7.1	Dataset Limitations	44
6.7.2	Single-Run Evaluation	45
6.7.3	External Validity	45
6.7.4	Stage Interaction Errors	45
6.8	Practical Applicability	45
6.9	Comparison to Prior Work	46
6.10	Future Work Directions	46
7	Conclusion	48
7.1	Summary	48
7.2	Research Questions Answered	49
7.3	Key Contributions	49
7.4	Limitations and Future Work	50
7.5	Final Remarks	50
	Bibliography	55
	Appendix A: Dataset Details	56
7.6	Stage-A Dataset Sources and Class Counts	56
7.7	Stage-B Dataset Sources and Class Counts	56
7.8	Stage-C Dataset Sources and Class Counts	56
7.9	Data Augmentation Settings	56
7.10	Stage-C Duplicate-Hash Groups	57
7.11	Metal Composition Table	58
7.12	Fallback Price Table	58
7.13	Metal Value Estimation Formula	58
	Appendix B: Training Configuration and Evidence Package	60
7.14	Final ViT Model Hyperparameters	60
7.15	Stage-B Anti-Overfitting Experiment Details	60
7.16	CNN Baseline Experiments	60
7.17	YOLO11x-cls Experiments	61
7.18	Overview	61
7.19	Key Output Files	61
7.20	Reproducing Training	62

List of Figures

3.1	Five-stage smartphone e-waste decision-support pipeline.	11
4.1	Frontend web interface — image upload page.	25
4.2	Frontend web interface — analysis results page (normal phone, resell recommendation).	25
5.1	Test accuracy comparison — CNN (DenseNet121), YOLO11x-cls, and ViT models across all stages.	30
5.2	Stage-A DINOv2 ViT-B/14 — test confusion matrix (absolute counts).	31
5.3	Stage-A DINOv2 ViT-B/14 — training and validation accuracy and loss curves.	31
5.4	Stage-A DINOv2 ViT-B/14 — validation weighted F1-score per epoch.	32
5.5	Stage-B Swin Transformer V2 Tiny — test confusion matrix (absolute counts).	33
5.6	Stage-B Swin V2 Tiny — training/validation accuracy and loss curves (24 epochs, early stopping at patience = 15).	34
5.7	Stage-C MaxViT-Tiny — test confusion matrix (absolute counts).	35
5.8	Stage-C MaxViT-Tiny — training/validation accuracy and loss curves (100 epochs, no early stopping).	35
5.9	Pipeline output screenshot for a normal phone with resell recommendation.	39
5.10	Pipeline JSON output example for a broken phone with minor damage (MetalpriceAPI live pricing source shown in <code>pricing_source</code> field).	40
5.11	Validation weighted F1-score curves — Stage-A (DINOv2), Stage-B (Swin V2), Stage-C (MaxViT).	41

List of Tables

3.1	Five-stage pipeline summary.	12
3.2	Dataset source types and primary application.	13
3.3	Stage-wise dataset summary.	14
3.4	Detailed class-level image counts by stage and split.	14
3.5	Model families and their roles in the study.	16
3.6	Training hyperparameter summary by model family.	17
3.7	Model selection criteria.	18
3.8	Estimated metal composition per smartphone model (corrected from source CSV).	18
3.9	Metal pricing sources and fallback values.	19
3.10	Stage-E rule-based recommendation logic.	20
4.1	Project directory structure and purpose.	22
4.2	Hardware and software environment.	23
4.3	FastAPI endpoint summary.	24
4.4	Stage-D MetalpriceAPI-enabled output fields (latest successful run). .	26
4.5	Representative final pipeline JSON outputs.	27
5.1	Final dataset statistics by stage.	28
5.2	Best CNN baseline results by stage.	28
5.3	YOLO11x-cls comparison results by stage.	29
5.4	Final selected ViT model results by stage.	29
5.5	Cross-family model comparison summary.	30
5.6	Stage-A classification report — DINOv2 ViT-B/14.	30
5.7	Stage-A confusion matrix — DINOv2 ViT-B/14 (test set).	31
5.8	Stage-B classification report — Swin Transformer V2 Tiny (WD = 0.05 run 20260609).	33
5.9	Stage-B confusion matrix — Swin Transformer V2 Tiny (test set, WD = 0.05 run 20260609).	33
5.10	Stage-C classification report — MaxViT-Tiny.	35
5.11	Stage-C confusion matrix — MaxViT-Tiny (test set).	35

5.12	Final five-stage pipeline summary.	36
5.13	Stage-D MetalpriceAPI pricing source validation.	36
5.14	Normal phone pipeline output example.	38
7.1	Stage-A dataset class counts by split.	56
7.2	Stage-B dataset class counts by split.	56
7.3	Stage-C dataset class counts by split.	57
7.4	Stage-C duplicate-hash group summary.	57
7.5	Stage-D metal composition database (grams per device).	58
7.6	Fallback metal price table (USD per gram).	58
7.7	Final ViT training hyperparameter summary.	60
7.8	CNN baseline training configuration.	61
7.9	YOLO11x-cls training configuration.	61
7.10	Evidence package contents.	62

Chapter 1

Introduction

1.1 Background of Smartphone E-Waste

Electronic waste is one of the fastest-growing waste streams globally. Modern consumer electronics contain a heterogeneous mixture of valuable metals, reusable components, hazardous materials, plastics, glass, and battery elements. Smartphones are particularly significant in this context because their replacement cycle is substantially shorter than that of most other electronic devices. A smartphone that is no longer in active use by its owner may still be suitable for resale, repair, refurbishment, parts recovery, or material recycling. The appropriate handling pathway depends on multiple interrelated factors: the device model, its visible physical condition, the severity of any damage, and the estimated recoverable material value [4], [53].

Traditional assessment of used smartphones is predominantly manual. A technician or recycler inspects the device visually, identifies the model, judges whether the device appears normal or damaged, estimates the level of damage, and then decides whether resale, repair, or recycling is the most appropriate course of action. This process is time-consuming, subjective, and difficult to scale when large volumes of devices must be sorted quickly. A computer vision system can support this process by providing consistent preliminary predictions directly from images, reducing repetitive manual work while leaving final decisions to human experts [1], [7].

Deep learning has matured into the dominant approach for visual classification. Convolutional neural networks established strong image recognition benchmarks, while more recent architectures such as Vision Transformers, Swin Transformers, MaxViT, and self-supervised representation models such as DINOv2 have further advanced performance across a wide range of visual tasks. Modern YOLO-family classification models also provide practical deployment-oriented comparisons. This thesis uses all three of these model families to study smartphone e-waste image

analysis in a structured, staged pipeline [17], [25].

1.2 Problem Statement

The central problem addressed by this thesis is the absence of an integrated image-based decision-support pipeline for smartphone e-waste assessment. Existing image classification studies typically identify a single object category but do not address the multi-stage nature of e-waste handling. A complete assessment requires identifying the device model, evaluating whether it appears normal or damaged, determining damage severity when the device is broken, estimating approximate recyclable metal value, and recommending a suitable handling pathway. Without this multi-stage structure, the output may be technically correct as a classification result but incomplete for practical recycling or resale decision-making.

The problem is therefore formulated as a five-stage pipeline. Given an input smartphone image, the system must predict the smartphone model, classify its condition, conditionally classify damage severity if the phone is broken, estimate recoverable metal value using a model-specific composition database and a hybrid API-plus-fallback pricing system, and generate a final recommendation. The solution must also be experimentally validated by comparing multiple deep learning approaches rather than assuming a single model family is optimal for all tasks.

1.3 Motivation

The motivation for this research is simultaneously environmental and technical. From an environmental perspective, improved e-waste sorting supports higher-value device reuse and more efficient material recovery. Reselling a device in normal condition preserves more economic and environmental value than immediately recycling it. Repairing a minor-damaged device extends its useful life. Recycling is more appropriate when a device is heavily damaged and repair is uneconomical. Each correctly made decision therefore reduces unnecessary material loss and environmental impact.

From a technical perspective, smartphone e-waste classification is a well-structured computer vision problem: model identification relies on recognising shape, camera layout, and device design cues; condition classification requires detecting visible damage or intactness; severity classification requires distinguishing minor from major damage patterns. Each sub-task is distinct enough to benefit from a dedicated model trained on a focused dataset, motivating a staged architecture over a single monolithic classifier.

An additional motivation is to demonstrate how deep learning outputs can be connected to practical decision logic. A model accuracy table alone does not show how a system supports an end user. This thesis therefore integrates classification models with a metal value estimation module powered by MetalpriceAPI and a web-based interface, so that the result can be interpreted as an end-to-end prototype while clearly stating its limitations.

1.4 Research Objectives

1. To develop an image-based smartphone model classification system for `iphone_8`, `iphone_x`, `iphone_11`, and `iphone_13`.
2. To develop a condition classification system that categorises smartphones as normal or broken.
3. To develop a damage severity classification system that categorises broken smartphones into `minor_damage` or `major_damage`.
4. To ensure that Stage-C executes only when Stage-B predicts the phone as broken.
5. To build a recyclable metal value calculation module using a metal composition database, MetalpriceAPI for live or delayed pricing, and a fallback CSV for unavailable metals or API failure.
6. To design a recommendation module that suggests resell-as-is, repair-then-resell, recycle, or manual review.
7. To compare CNN-based models, YOLO11x-cls models, and Vision Transformer-based models across the three classification stages.
8. To integrate the final selected pipeline into a web-based demonstration system using a Python FastAPI backend and a React frontend.

1.5 Research Questions

- RQ1** How accurately can a deep learning model classify smartphone models from images in the custom-curated dataset?
- RQ2** How accurately can a model distinguish normal and broken smartphone conditions from image evidence?

- RQ3** Can a dedicated damage severity stage improve the interpretation of broken devices by classifying minor and major damage?
- RQ4** How do CNN, YOLO11x-cls, and Vision Transformer-based approaches compare across the three classification stages?
- RQ5** Can predicted smartphone model information be connected to a metal composition database to estimate recyclable metal value using live market prices from MetalpriceAPI?
- RQ6** Can the classification and metal value outputs be integrated into a decision-support recommendation for e-waste handling?

1.6 Scope of the Study

The study is limited to image-based analysis of four selected iPhone models: `iphone_8`, `iphone_x`, `iphone_11`, and `iphone_13`. The condition classes are limited to normal and broken, and the damage severity classes are limited to `minor_damage` and `major_damage`. The system does not perform electrical testing, battery health diagnosis, screen functionality checking, internal component inspection, or industrial disassembly analysis.

The recyclable metal value is estimated from a device-specific composition database combined with MetalpriceAPI pricing; it should be interpreted as a research-level estimate and not as a certified recycling quotation or industrial-grade valuation. The deployed demonstration is a prototype system. Results are valid for the prepared dataset and the specific experimental configuration documented in the evidence package. Further evaluation on larger, more diverse, real-world datasets is required before any practical deployment.

1.7 Expected Contributions

- A five-stage smartphone e-waste analysis pipeline combining model classification, condition assessment, severity classification, metal value estimation, and recommendation.
- A custom-curated smartphone e-waste dataset collected from mixed online and real-world sources.
- A comparative analysis of CNN baseline models, YOLO11x-cls classification models, and Vision Transformer-based models.

- A final deployed ViT-based pipeline using DINOv2 ViT-B/14, Swin Transformer V2 Tiny, and MaxViT-Tiny.
- A hybrid metal value estimation module integrating MetalpriceAPI with a fallback CSV price table.
- A FastAPI/React web application demonstration connecting model inference with user-facing output.
- A documented evidence package containing datasets, metrics, confusion matrices, training curves, JSON outputs, code files, and environment information.

1.8 Thesis Organisation

Chapter 2 reviews related work in e-waste management, computer vision for waste classification, CNN, YOLO, Vision Transformers, DINOv2, Swin Transformer V2, MaxViT, metal value estimation, and decision-support systems. Chapter 3 describes the methodology in full detail, including dataset construction, preprocessing, augmentation, training configuration, experimental design, evaluation metrics, and the MetalpriceAPI integration. Chapter 4 explains implementation details including the pipeline script, FastAPI backend, frontend interface, GPU environment, and Stage-D hybrid pricing system. Chapter 5 presents results and analysis using dataset statistics, CNN/YOLO/ViT comparisons, final selected model tables, confusion matrices, training curves, Stage-D live pricing validation, and pipeline output examples. Chapter 6 discusses interpretation, limitations, misclassification causes, and practical usefulness. Chapter 7 concludes the thesis and presents future work directions.

Chapter 2

Literature Review

2.1 E-Waste and Smartphone Recycling

Electronic waste management is an interdisciplinary problem that spans environmental science, resource recovery, public policy, logistics, and technology. The Global E-waste Monitor reports that e-waste generation has grown consistently year-on-year, with only a fraction of discarded electronics processed through formal recycling channels. Smartphones contain valuable materials including gold, silver, copper, aluminium, and cobalt, as well as plastics, glass, and battery-related compounds. The presence of both commercially valuable and potentially hazardous materials makes proper handling critical from both economic and environmental perspectives [53].

Manual e-waste sorting typically relies on workers visually identifying device type and condition, a process that can vary considerably across workers and locations. Computer vision can support the early assessment process by producing consistent preliminary labels. However, any responsible system must be clear about its limitations: image-based classification cannot determine internal defects, battery health, display functionality, network lock status, or certified market value. The literature therefore supports framing visual AI as a decision-support tool rather than a complete replacement for specialist inspection [1].

Studies on e-waste management show that informal handling, poor collection systems, and inconsistent policy enforcement can increase environmental and health risks [10], [14], [45]. At the same time, value-recovery studies show that electronic products contain recoverable metals whose economic and environmental value depends on collection, disassembly, recovery efficiency, and refining processes [2], [6], [13], [15].

Manufacturer environmental reports also show why smartphone assessment should be model-aware. Apple reports for iPhone 8, iPhone X, iPhone 11, and iPhone 13 describe device-level environmental attributes, material efficiency, recyclability, and

product-life-cycle assumptions [21], [22], [29], [38]. These reports do not provide a complete industrial recovery model for discarded phones, but they support the broader point that smartphones are material products with model-specific design and material profiles.

2.2 Computer Vision in Waste Management

Computer vision has been applied to waste sorting, recyclable material identification, bin monitoring, and conveyor-based material separation. Key advantages include processing speed and consistency once the model is trained. The main challenge is dataset variability: waste images vary widely in lighting, angle, occlusion, background, and damage state. Single-label classification systems that identify one waste category are common in the literature, but e-waste analysis requires answering multiple questions about the same object: what model is it, is it damaged, how severe is the damage, and what action should be taken. This motivates a multi-stage architecture [23], [36], [46].

General waste-classification work has used image datasets such as TrashNet and CNN-based classifiers to separate recyclable categories [20], [28]. E-waste-specific work has recently explored transformer-based classification, including EWasteNet for an E-Waste Vision Dataset [52]. These works show that image-based classification can support sorting tasks, but many systems stop at a single object or waste-category label. The gap in this thesis is the connection between visual classification and downstream smartphone e-waste decisions.

Multi-stage design allows each model to specialise on a narrower task and supports conditional execution. The most important conditional rule in this thesis is that Stage-C is not executed for devices predicted as normal, avoiding irrelevant severity labels and ensuring the recommendation logic receives consistent inputs. This staged approach has been used in related multi-step inspection and quality assessment systems in the computer vision literature [52].

2.3 CNN-Based Image Classification

Convolutional neural networks are the foundational architecture for deep learning image classification. CNNs apply learned convolutional filters across an image to extract spatial patterns, edges, textures, shapes, and higher-level features. Landmark architectures include AlexNet [12], VGG [16], ResNet [17], DenseNet [25], MobileNet [24], [27], EfficientNet [33], and ConvNeXt [47]. Their inductive bias toward local spatial structure makes them effective across a broad range of image datasets.

In this thesis, CNN-based models serve as baseline experiments. Baselines are necessary because they provide a reference point for evaluating more advanced architectures. DenseNet121 was selected as the primary CNN baseline because of its strong parameter efficiency and feature reuse through dense connections. If transformer-based models produce a clear performance improvement over CNN baselines, then selecting them for deployment is experimentally defensible [17], [25].

2.4 YOLO Classification Models

YOLO architectures are widely associated with real-time object detection, but recent YOLO implementations from Ultralytics include dedicated classification modes. YOLO classification models benefit from practical tooling, an efficient training workflow, and fast inference [18], [35]. In this thesis, YOLO11x-cls was evaluated as a modern, practical classification comparison rather than as an object detector. The purpose was to compare a state-of-the-art practical classification model against CNN baselines and Vision Transformer models on the same stage-wise datasets. The YOLO experiments provide an important middle ground: stronger than simple CNN baselines but evaluated against the final ViT models for each stage [54], [59].

2.5 Vision Transformers

Vision Transformers (ViT) adapt the transformer architecture to image tasks by dividing images into fixed-size patches and processing them through self-attention mechanisms [40]. Unlike CNNs, which embed local spatial priors, transformers can model long-range relationships across distant image regions. This is beneficial for tasks where global design cues matter: smartphone model classification may benefit from global recognition of camera layout, body geometry, and notch design; damage severity classification may benefit from both local damage patterns and broader contextual assessment.

DeiT improved data-efficient training for image transformers [44], on the other hand Swin Transformer introduced hierarchical shifted-window attention for vision backbones [42]. Swin Transformer V2 addressed scaling and resolution-transfer challenges [48]. MaxViT combines blocked local and dilated global attention to capture multi-axis interactions [50]. DINO and DINOv2 show the value of self-supervised visual representations for robust feature extraction [39], [55].

This thesis evaluates three Vision Transformer-related architectures: DINOv2 ViT-B/14, Swin Transformer V2 Tiny, and MaxViT-Tiny. These represent distinct design ideas within the transformer family and were selected based on their respective task suitability and final test performance across the three classification stages.

2.6 DINOv2, Swin Transformer V2, and MaxViT

DINOv2 [55] is a self-supervised visual representation learning framework that produces strong general-purpose features without relying on manually labelled pretraining data. The DINOv2 ViT-B/14 model provides a 768-dimensional patch-level CLS token that encodes rich visual representations. In this thesis, DINOv2 was selected for Stage-A because model classification requires robust global features to discriminate between visually similar iPhone generations.

Swin Transformer V2 [48] introduces hierarchical feature maps and shifted-window self-attention with improved positional encoding and resolution scaling. Its compact Tiny variant was selected for Stage-B because condition classification benefits from multi-scale feature representation that can detect subtle damage patterns while remaining computationally efficient. The Tiny variant balances accuracy with manageable model size for the binary classification task.

MaxViT [50] combines multi-axis attention with convolutional blocks to capture both local and global visual information within a unified architecture. Its window-based and dilated attention mechanisms make it suitable for tasks where both fine-grained local cues and broader structural context matter. MaxViT-Tiny was selected for Stage-C damage severity classification, where the distinction between minor and major damage can depend on crack extent, surface area, and structural deformation visible across different scales.

2.7 MetalpriceAPI and Metal Value Estimation

Metal value estimation connects computer vision outputs to recycling decision support. A predicted smartphone model can be mapped to an approximate device composition and estimated recoverable amounts of specific metals. By multiplying each metal amount by its market price per gram and a recovery rate, the system estimates a total recyclable material value. The output currency in this thesis is Bangladeshi Taka (BDT), obtained by converting USD-based metal prices using a real-time or delayed exchange rate from MetalpriceAPI [3], [15].

MetalpriceAPI provides market-based prices for precious and industrial metals. The API returns prices in USD per troy ounce for precious metals (XAU for gold, XAG for silver) and USD per standard ounce for industrial metals (XCU for copper, ALU for aluminium, XCO for cobalt). The API also returns currency exchange rates including USD-to-BDT. This thesis uses MetalpriceAPI as the primary pricing source and retains a fallback CSV table for metals with no API symbol, unsupported metals such as titanium and rare earth elements, or in the event of API failure or network unavailability.

The metal value module is intentionally described as an estimate. Industrial recycling valuation depends on many factors not modelled here, including actual material recovery efficiency, device-specific internal condition, battery disassembly cost, refining losses, labour cost, transportation, and spot market volatility. The estimated value is therefore a decision-support signal rather than a financially binding recycling quotation.

2.8 Evaluation, Reproducibility, and Dataset Documentation

Classification results should be reported with more than accuracy because class supports and error types matter [5], [9]. Machine-learning studies should also document software, data splits, limitations, and risks of leakage [11], [43]. Dataset and model documentation practices such as datasheets and model cards provide useful guidance for transparent reporting [31], [41]. This thesis adopts that principle by reporting the duplicate-hash leakage risk and by acknowledging dataset limitations rather than filling them with unsupported claims.

2.9 Research Gap

The research gap addressed by this thesis is the absence of an integrated smartphone e-waste assessment pipeline that combines multi-stage image classification, live-market metal value estimation, and rule-based recommendation in a single deployable prototype. Existing image classification studies typically stop at a single class label. E-waste handling requires a more complete interpretation: identifying the model, assessing condition, evaluating damage severity, estimating material value, and recommending an action pathway. The combination of these stages in a single, experimentally evaluated prototype with MetalpriceAPI integration represents the novelty of this work.

Chapter 3

Methodology

3.1 Full Five-Stage System Overview

The proposed methodology is structured as a five-stage pipeline that processes a single smartphone image through consecutive specialised modules, each contributing a distinct type of output toward a final handling recommendation. The sequential nature of the pipeline ensures that downstream stages receive contextualised inputs: Stage-C receives only images that Stage-B has already classified as broken, and Stage-D receives the predicted model class from Stage-A to select the correct metal composition record.

Stage-A performs smartphone model classification and outputs one of four class labels: `iphone_8`, `iphone_x`, `iphone_11`, or `iphone_13`. Stage-B performs binary condition classification, outputting either `normal` or `broken`. Stage-C is conditionally executed: it runs only when Stage-B predicts `broken`, and it classifies damage severity as either `minor_damage` or `major_damage`. Stage-D estimates recyclable metal value in Bangladeshi Taka using a model-specific composition database and a hybrid pricing system combining MetalpriceAPI live or delayed market data with a fallback CSV. Stage-E applies rule-based decision logic to generate the final recommendation. The full workflow is illustrated in 3.1.

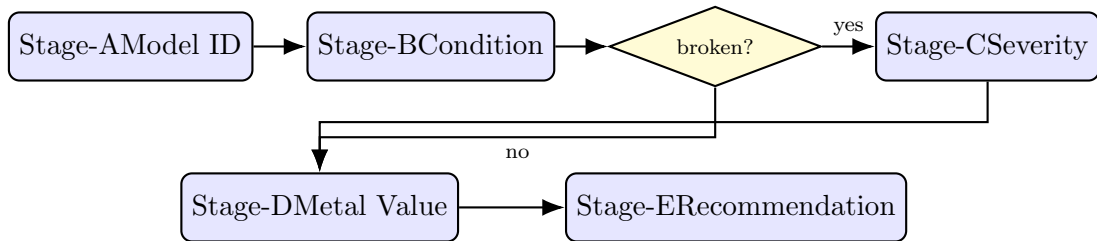


Figure 3.1: Five-stage smartphone e-waste decision-support pipeline.

Table 3.1: Five-stage pipeline summary.

Stage	Task	Input	Output	Execution Condition
A	Smartphone model classification	Smartphone image	iphone.8 / iphone_x / iphone.11 / iphone.13	Always
B	Condition classification	Smartphone image	normal / broken	Always
C	Damage severity classification	Smartphone image	minor_damage / major_damage	Only if Stage-B predicts broken
D	Recyclable metal value calculation	Stage-A predicted model	Estimated value in BDT	Always
E	Recommendation	Stage-B and Stage-C outputs	Resell / Repair / Recycle / Manual review	Always

3.2 Custom Dataset Construction Overview

No pre-existing, publicly available dataset was found that simultaneously covers smartphone model identification, normal versus broken condition classification, and minor versus major damage severity classification for the four iPhone models used in this thesis. Consequently, a custom dataset was constructed and curated specifically for this research. The dataset is the result of combining images from multiple online sources with a set of original real-world images collected through direct visits to local phone repair and scrap/vangari shops.

The total dataset comprises three independent stage-wise sub-datasets. Stage-A contains 4,000 images across four model classes. Stage-B contains 8,000 images across two condition classes. Stage-C contains 3,097 images across two damage severity classes. Each sub-dataset has a dedicated train, validation, and test split with no overlap between splits for the same stage, ensuring unbiased evaluation.

3.3 Dataset Sources

3.3.1 Online Image Collection

A substantial portion of the dataset was collected from publicly accessible online sources including Google Images, Facebook Marketplace, Pinterest, and product review and resale websites. Search queries were constructed using specific iPhone model names combined with condition-related terms such as “broken screen”, “cracked”, “damaged”, “used”, “mint condition”, and “scratched”. Downloaded images were screened manually or semi-manually for relevance: images clearly showing

the target device model and the correct condition category were retained, while off-topic images were discarded.

3.3.2 Real-World Data Collection from Repair and Scrap Shops

To supplement online sources and improve the representation of real-world device conditions, original images were collected by visiting local phone repair shops and scrap/vangari shops. At these locations, damaged and discarded smartphones were photographed directly, providing images that reflect genuine real-world damage patterns, varied lighting, and authentic device states. All images collected from these real-world visits were original and captured specifically for this dataset.

Table 3.2: Dataset source types and primary application.

Source Type	Description	Primary Use
Google Images	Search queries combining model names with condition keywords	Stage-A, Stage-B, Stage-C
Facebook Marketplace	Listings of used and damaged smartphones	Stage-B, Stage-C
Pinterest	Images of damaged and intact devices	Stage-A, Stage-B
Product/resale websites	Device images from e-commerce and recycling platforms	Stage-A, Stage-B
Phone repair shops (local)	Directly photographed broken devices and screens	Stage-B, Stage-C
Scrap/vangari shops (local)	Directly photographed heavily damaged and discarded devices	Stage-C, Stage-B

3.4 Stage-Wise Dataset Design

3.4.1 Stage-A Dataset: Smartphone Model Classification

The Stage-A dataset contains images of four iPhone model classes. These four models represent distinct iPhone design generations spanning different form factors, camera configurations, display notch styles, and body dimensions. Stage-A images include both normal and broken devices, since model classification is required regardless of condition. The total Stage-A dataset comprises 3,036 training images, 482 validation images, and 482 test images.

3.4.2 Stage-B Dataset: Normal vs. Broken Condition Classification

The Stage-B dataset is a binary classification dataset. The training split contains a near-balanced distribution of broken and normal images: 3,550 broken and 3,486 normal training images. The Stage-B dataset contains 7,036 training images, 482 validation images, and 482 test images.

3.4.3 Stage-C Dataset: Minor vs. Major Damage Severity Classification

The Stage-C dataset contains only broken smartphone images. The Stage-C dataset contains 2,197 training images, 450 validation images, and 450 test images. The validation and test splits contain exactly 225 images per class, making Stage-C evaluation balanced.

Table 3.3: Stage-wise dataset summary.

Stage	Task	Train	Val	Test	Total	Classes
A	Model classification	3,036	482	482	4,000	4
B	Condition classification	7,036	482	482	8,000	2
C	Damage severity classification	2,197	450	450	3,097	2

Table 3.4: Detailed class-level image counts by stage and split.

Stage	Class	Train	Val	Test
Stage-A	iphone_11	794	103	103
Stage-A	iphone_13	794	103	103
Stage-A	iphone_8	744	128	128
Stage-A	iphone_x	704	148	148
Stage-B	broken	3,550	225	225
Stage-B	normal	3,486	257	257
Stage-C	major_damage	1,057	225	225
Stage-C	minor_damage	1,140	225	225

3.5 Class Definitions and Labelling Rules

3.5.1 Smartphone Model Label Criteria

Smartphone model labelling was based on visual identity. `iphone_8` images show a device with a physical home button and a single rear camera. `iphone_x` images show a Face ID notch and a vertical dual rear camera. `iphone_11` images show a Face ID

notch with a dual camera in a square housing. `iphone_13` images show a smaller notch and a diagonal dual camera arrangement distinguishable from the `iphone_11`.

3.5.2 Normal and Broken Class Criteria

The *normal* class contains images showing no visible screen cracks, no significant body damage, no obvious frame deformation, and no shattered glass. The *broken* class contains images showing at least one of: visible screen cracks, broken back glass, cracked or missing components, frame damage or bending, or shattered display glass.

3.5.3 Minor and Major Damage Class Criteria

The *minor_damage* class covers broken devices with limited, localised visible damage such as small hairline cracks confined to a corner of the screen, minor scratches, small chips on the frame, or isolated damage patterns not compromising most of the screen area.

The *major_damage* class covers broken devices with extensive visible damage such as cracks extending across the full screen, shattered displays with significant glass missing, large structural deformations, screen separation, or structural failures affecting multiple components simultaneously.

3.5.4 Borderline Case Handling

Borderline cases were handled by assigning the image to the category that the damage pattern most strongly suggested, or by removing the image if the correct class could not be determined confidently.

3.6 Data Cleaning and Preprocessing

All images were validated using PIL/Pillow, converted to RGB format, and resized to the input resolution required by each model using bilinear interpolation. Non-image files were excluded, and corrupted images were removed. Input resolutions were set to 224×224 pixels for CNN, DINOv2, and MaxViT, and 256×256 pixels for SwinV2 Tiny, following the official pretrained model configurations.

Images were converted to tensors using `torchvision.transforms.ToTensor()`, which scales pixel values from $[0, 255]$ to $[0, 1]$. Subsequently, ImageNet normalization was applied using a mean of $[0.485, 0.456, 0.406]$ and a standard deviation of $[0.229, 0.224, 0.225]$.

3.7 Data Augmentation Strategy

Augmentation was applied exclusively to the training split. The augmentation pipeline included: random horizontal flip ($p=0.5$), random rotation ($\pm 15^\circ$), RandomResizedCrop (scale 0.7–1.0), ColorJitter (brightness, contrast, saturation ± 0.2), random affine transformations (translation up to 10%), and Gaussian blur ($p=0.1$). Validation and test splits were kept unaugmented to ensure reproducible evaluation.

3.8 Dataset Splitting Strategy

Each stage-wise dataset was split into non-overlapping training (approx. 75%), validation (approx. 12.5%), and test (approx. 12.5%) partitions. Stratified splitting maintained consistent class distributions. No image file was permitted to appear in more than one split within the same stage. Cross-stage duplication between Stage-B and Stage-C was also checked.

3.9 Model Families Tested

Three model families were evaluated. CNN-based models provide conventional deep learning baselines. YOLO11x-cls provides a practical deployment-oriented comparison pipeline. Vision Transformer models provide the final deployed model family.

Table 3.5: Model families and their roles in the study.

Family	Model	Library	Pretrained	Role
CNN	DenseNet121	torchvision	ImageNet-1K	Baseline
YOLO	YOLOv11x-cls	Ultralytics	ImageNet-1K	Practical comparison
ViT (self-sup.)	DINOv2 ViT-B/14	HF Transformers	LVD-142M	Final Stage-A
ViT (hierarchical)	Swin Transformer V2 Tiny	timm	ImageNet-1K	Final Stage-B
ViT (multi-axis)	MaxViT-Tiny	timm	ImageNet-1K	Final Stage-C
ViT (distillation)	DeiT-Small	timm	ImageNet-1K	ViT comparison
ViT (enhanced)	EVA-02-B	timm	IN-22K+1K	ViT comparison

3.10 Transfer Learning and Fine-Tuning

All models were initialised from pretrained weights and fine-tuned on stage-specific datasets. DINOv2 ViT-B/14 was loaded via the Hugging Face transformers Auto-Model interface; a classification head consisting of Dropout($p=0.4$) followed by a Linear layer was appended to the 768-dimensional CLS token. Swin Transformer V2 Tiny and MaxViT-Tiny were loaded via the timm library; pretrained classification heads were replaced with stage-specific output layers. All models were fine-tuned end-to-end using AdamW.

For the initial training phase of some experiments, backbone layers were selectively frozen for a warm-up period (typically 5 epochs) before end-to-end fine-tuning.

3.11 Training Configuration

AdamW was used as the primary optimiser ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$), consistent with best practices for transformer fine-tuning [30]. A cosine annealing learning rate scheduler was applied in most configurations [26]. Label smoothing of 0.1 was used in selected configurations [19]. Mixed-precision training (AMP) was enabled to reduce GPU memory usage.

3.6 summarises the final training hyperparameters used for each model family.

Table 3.6: Training hyperparameter summary by model family.

Parameter	CNN Baseline	YOLOv11x-cls	DINOv2 ViT-B/14	Swin V2	MaxViT-Tiny
Optimiser	Adam	SGD (default)	AdamW	AdamW	AdamW
Learning rate	1e-4	default	2e-05	2e-05	2e-05
Batch size	32	default	16	16	16
Image size (px)	224	224	224	256	224
Weight decay	1e-4	default	1e-4	1e-4	1e-4
Dropout	0.4	N/A	0.4	0.4	0.5
Label smoothing	0.0	N/A	0.1	0.1	0.1
Scheduler	StepLR	cosine	cosine	cosine	cosine

3.12 Evaluation Metrics

The classification stages are evaluated using accuracy, weighted precision, weighted recall, macro-F1, weighted-F1, classification reports, and confusion matrices. For class i , precision and recall are:

$$\text{Precision}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i}, \quad \text{Recall}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i}. \quad (3.1)$$

The per-class F1-score is:

$$F1_i = 2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}. \quad (3.2)$$

Weighted-F1 is:

$$\text{Weighted F1} = \frac{\sum_c F1_c \times \text{support}_c}{\sum_c \text{support}_c}. \quad (3.3)$$

Weighted-F1 accounts for class support and is the primary metric for model comparison [5], [9]. Macro-F1 treats all classes equally and is reported alongside weighted-F1 to detect per-class weaknesses.

3.13 Model Selection Criteria

The final model for each stage was selected based on test weighted F1-score (primary), then test accuracy, then validation F1. Secondary criteria included confusion matrix behaviour (particularly broken predicted as normal in Stage-B), training stability, and practical integrability into `run_vit_pipeline.py`.

Table 3.7: Model selection criteria.

Criterion	Weight	Description
Test weighted F1	Primary	Main ranking metric per stage
Test accuracy	Primary	Equal weight to F1
Validation weighted F1	Secondary	Best checkpoint selection
Confusion matrix behaviour	Secondary	Critical error types
Training stability	Informational	Training/validation curves
Pipeline integrability	Mandatory	Must run in <code>run_vit_pipeline.py</code>

3.14 Stage-D Metal Value Estimation Methodology

3.14.1 Metal Composition Database

Stage-D uses a CSV-based metal composition database that maps each supported smartphone model to estimated recoverable amounts of selected metals in grams. The composition entries include gold, silver, copper, aluminium, and cobalt.

Table 3.8: Estimated metal composition per smartphone model (corrected from source CSV).

Model	Weight (g)	Gold (g)	Silver (g)	Copper (g)	Aluminium (g)	Cobalt (g)
iphone.8	148	0.018	0.185	11.75	22.0	4.25
iphone.x	174	0.021	0.24	13.5	31.0	5.25
iphone.11	194	0.024	0.27	15.75	35.0	7.0
iphone.13	174	0.021	0.24	14.75	29.0	7.5

3.14.2 MetalpriceAPI Integration

The primary pricing source in Stage-D is MetalpriceAPI, a commercial metal price data service providing live or delayed market-based prices. The API endpoint used is <https://api.metalpriceapi.com/v1/latest> with base currency USD. Metal symbols used are: XAU (gold), XAG (silver), XCU (copper), ALU (aluminium), and XCO (cobalt). The pipeline output includes the field `pricing_source` (metalpriceapi_with_fallback when API succeeds, or `{fallback_only_api_failed}` when it fails).

3.14.3 USD-to-BDT Conversion

The USD-to-BDT exchange rate is obtained from MetalpriceAPI. If the API is unavailable, the fallback exchange rate from `currency_config.json` is used. MetalpriceAPI returns precious metal prices in USD per troy ounce (1 troy oz = 31.1035 g) and industrial metal prices in USD per standard ounce (1 oz = 28.3495 g). The estimated recyclable value is:

$$V = \sum_{m \in M} a_m \times r_m \times p_m \times c, \quad (3.4)$$

where a_m is metal amount (g), r_m is recovery rate, p_m is price (USD/g), and c is the USD-to-BDT exchange rate.

Table 3.9: Metal pricing sources and fallback values.

Metal	API Symbol	Unit	Approx. price (USD/g)	Notes
Gold	XAU	troy_oz	~139.44	Primary: MetalpriceAPI; Fallback: CSV
Silver	XAG	troy_oz	~2.22	Primary: MetalpriceAPI; Fallback: CSV
Copper	XCU	oz	~0.0141	Primary: MetalpriceAPI; Fallback: CSV
Aluminium	ALU	oz	~0.0036	Primary: MetalpriceAPI; Fallback: CSV
Cobalt	XCO	oz	~0.0563	Primary: MetalpriceAPI; Fallback: CSV
Titanium	N/A	N/A	~0.006	No API symbol; fallback only
Rare earths	N/A	N/A	~0.02	No API symbol; fallback only

3.14.4 Fallback Price Mechanism

The fallback price table (`metal_price_fallback.csv`) provides price estimates for metals unavailable through MetalpriceAPI or in case of API failure. Fallback prices are used only when the API does not provide a price for a specific metal; they do not override API prices when the API succeeds.

3.14.5 Limitations of Estimated Recyclable Value

The Stage-D estimate has several explicitly stated limitations: industrial recovery rates may differ from database values; API prices may be delayed rather than real-time; the estimate excludes disassembly, refining costs, labour, transportation, and battery handling fees. The estimated value is a research-level decision-support signal.

3.15 Stage-E Recommendation Methodology

Stage-E converts the outputs of Stages A, B, and C into a final handling recommendation using deterministic rule-based decision logic.

Table 3.10: Stage-E rule-based recommendation logic.

Stage-B	Stage-C	Recommendation	Code	Reason
normal	not executed	Resell as-is	resell_as_is	Normal device is more suitable for resale before recycling.
broken	minor_damage	Repair then resell	repair_then_resell	Minor damage may preserve economic value through repair.
broken	major_damage	Recycle	recycle	Major damage makes repair or resale less economically viable.
uncertain	unavailable	Manual review required	manual_review	Human inspection required.

3.16 Reproducibility

Training experiments were performed on an NVIDIA GeForce RTX 5090 GPU with 32,607 MiB total GPU memory. PyTorch 2.11.0+cu128 was used with CUDA 12.8 and Python 3.11. Key libraries included: transformers (HuggingFace), timm, ultralytics (YOLO11), fastapi, pillow, scikit-learn [8], [32], [34]. A random seed was set at the beginning of each training script using `torch.manual_seed()` to improve reproducibility; however, full determinism is not guaranteed due to GPU atomic operations.

3.17 Threats to Validity

Internal validity threats include inconsistent hyperparameter configurations across model families and GPU non-determinism. External validity concerns include the custom-curated dataset’s mixed sources which may not fully represent all real-world smartphone conditions. Construct validity concerns include the metal value estimate not being independently validated against industrial recycling data. Dataset bias may arise from overrepresentation of specific visual styles from online sources and subjective labelling for borderline severity cases.

3.18 Dataset Limitations

The dataset is limited to four iPhone models. Class sizes are relatively small compared to large-scale industrial datasets. Real-world repair and scrap shop images represent only a subset of the total dataset. Label consistency for borderline cases cannot be fully guaranteed. The dataset does not include multi-angle views per device. External validation on an independently collected test set was not performed.

Chapter 4

Implementation

4.1 Project Architecture Overview

The implementation is organised into a structured directory hierarchy that maps each component of the thesis to a specific evidence folder. The project root contains the dataset directory, model checkpoint directory, source code, results directory, output directory, and the web application subdirectory.

Table 4.1: Project directory structure and purpose.

Folder / File	Purpose
database/ewaste_thesis_database/	Stage-A/B/C datasets and Stage-D metal database
models/final/	Final ViT model checkpoints for Stages A, B, C
models/experiments/	All experiment checkpoints (CNN, YOLO, ViT)
results/	Metrics CSVs, confusion matrices, training history
outputs/	Final pipeline JSON outputs and HTML reports
run_vit_pipeline.py	Final ViT end-to-end pipeline script with MetalpriceAPI
api_server.py / ewaste_thesis_website/	FastAPI backend and React frontend
src/	Training scripts for CNN, YOLO, and ViT models
requirements.txt	Python package requirements

4.2 Final ViT Pipeline Script: run_vit_pipeline.py

The final end-to-end pipeline is implemented in `run_vit_pipeline.py`. This script orchestrates all five stages of inference for a single input image. It implements a unified checkpoint loading mechanism that handles both DINOv2 ViT-B/14 (loaded

Table 4.2: Hardware and software environment.

Component	Configuration
GPU	NVIDIA GeForce RTX 5090
GPU Memory	32,607 MiB total
Driver	596.21
CUDA	12.8
PyTorch	2.11.0+cu128
Python	3.11
OS	Windows 11

via HuggingFace transformers) and timm-based models (Swin V2 Tiny, MaxViT-Tiny) through a common interface [34], [37].

4.2.1 Stage-B Conditional Logic for Stage-C

The conditional execution of Stage-C is implemented as a simple if-else block. After Stage-B predicts the device condition, the value is checked: if it equals "broken", Stage-C is executed on the same input image. If Stage-B predicts "normal", Stage-C is skipped and the `stage_c_output` dictionary is populated with `executed=False` and null values.

Listing 4.1: Stage-B conditional logic for Stage-C execution.

```

1 if predicted_condition == "broken":
2     stage_c_bundle = load_vit_classifier(stage_c_model_path, device
3     )
4     stage_c_pred = predict_vit(image_path, stage_c_bundle, device)
5     predicted_severity = stage_c_pred["predicted_class"]
6     stage_c_output["executed"] = True
7     stage_c_output["predicted_severity"] = predicted_severity
8 else:
9     warnings.append("Stage-C was skipped because Stage-B predicted
10     normal.")

```

4.2.2 MetalpriceAPI Integration in Stage-D

Stage-D calls `calculate_recyclable_value()` with the predicted model name, the database root path, and the `price_mode` argument. When `price_mode="auto"` (the default), the function calls `fetch_metalpriceapi_latest()` to retrieve live or delayed market prices. The Stage-D output includes the fields `pricing_source`, `metalpriceapi.success` (boolean), `metalpriceapi.successful_symbols` (list), and `usd_to_bdt` (numeric). When the API succeeds for at least one metal, `pricing_source` is set to `metalpriceapi.with_fallback`; on full failure it is set to `fallback_only_api_failed`.

4.2.3 Output Structure

The pipeline produces a structured JSON result containing: `image_path`, `created_at` timestamp, `pipeline_type`, stage-wise prediction dictionaries, metal breakdown list, pricing metadata, recommendation fields, and a warning string. The JSON output is saved to the `outputs/` directory with a timestamp-based filename.

4.3 FastAPI Web API

The web API is implemented using FastAPI and provides three endpoints [58]. CORS middleware is enabled to allow the React frontend to communicate with the backend during local demonstration.

Table 4.3: FastAPI endpoint summary.

Endpoint	Method	Purpose
/	GET	Returns API metadata and active pipeline description
/health	GET	Checks model file existence and environment configuration
/predict	POST	Accepts image upload, runs pipeline, returns JSON result
/docs	GET	FastAPI auto-generated Swagger UI documentation

4.4 React Frontend Interface

The frontend is a React-based web interface that provides an image upload form and a result display panel [57]. The frontend displays: predicted smartphone model and confidence, condition classification and confidence, damage severity and confidence (or “skipped” for normal devices), estimated recyclable value in BDT, MetalpriceAPI success status, USD-to-BDT rate and source, per-metal breakdown, and the final recommendation label and reason.

4.5 Training Scripts

Separate training scripts were implemented for CNN baseline experiments, YOLO classification experiments, and ViT experiments. The ViT training script includes a `MODEL_REGISTRY` dictionary mapping model keys to their HuggingFace or timm identifiers, recommended image sizes, and loading configurations. Model checkpoints include all metadata needed to reconstruct the model at inference time.

4.6 Stage-D Database Files

The Stage-D database is stored in the folder:

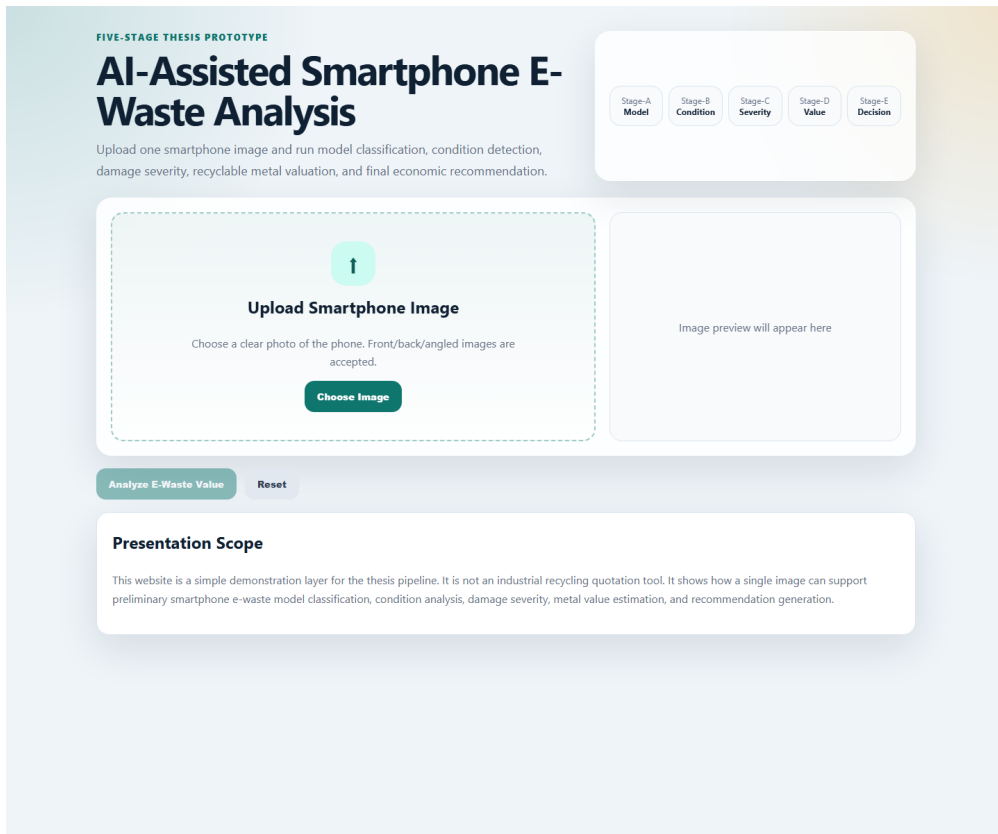


Figure 4.1: Frontend web interface — image upload page.

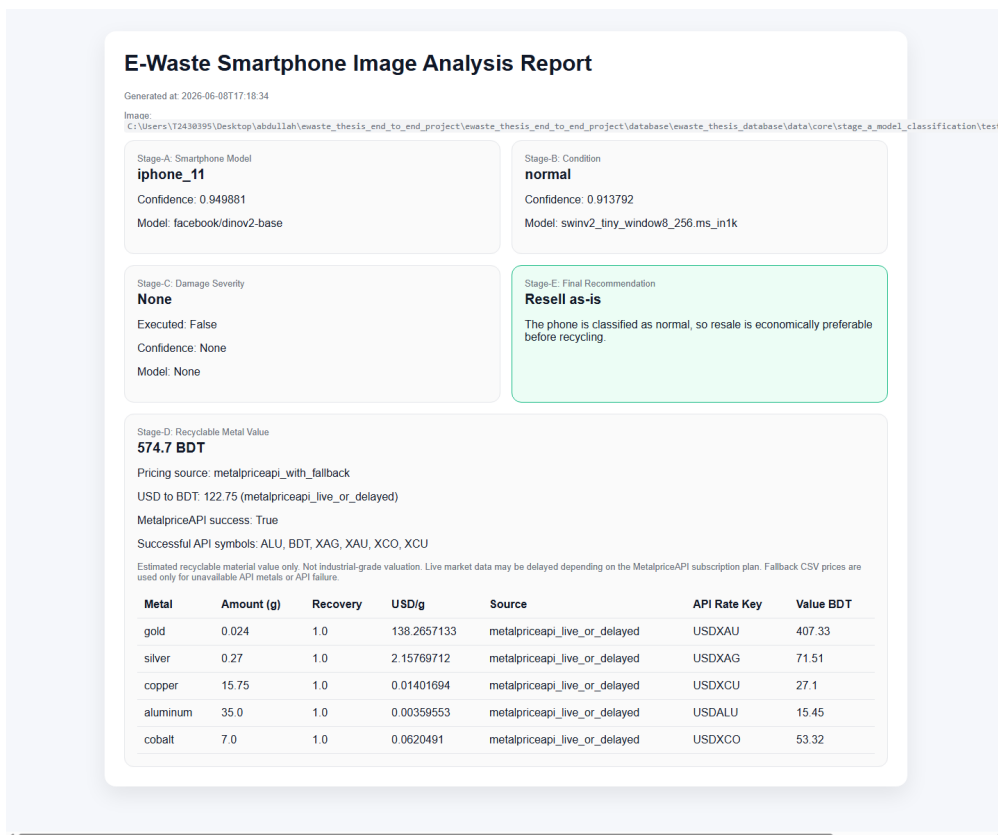


Figure 4.2: Frontend web interface — analysis results page (normal phone, resell recommendation).

database/ewaste_thesis_database/data/core/stage_d_metal_value/

and contains:

- `metal_composition_db.csv` — Maps each iPhone model to estimated metal amounts in grams.
- `metal_price_fallback.csv` — Fallback USD-per-gram prices.
- `currency_config.json` — Fallback USD-to-BDT exchange rate.
- `live_price_cache.json` — Cached MetalpriceAPI response (refreshed every 3,600 seconds).

4.7 Pipeline Output: MetalpriceAPI Enabled Run

The latest successful Stage-D run using MetalpriceAPI confirmed that all five metal symbols and the USD-to-BDT exchange rate were successfully retrieved.

Table 4.4: Stage-D MetalpriceAPI-enabled output fields (latest successful run).

Field	Value / Description
<code>pricing_source</code>	<code>metalpriceapi_with_fallback</code>
<code>metalpriceapi.success</code>	true
<code>metalpriceapi.successful_symbols</code>	ALU, BDT, XAG, XAU, XCO, XCU
<code>metalpriceapi.failed_symbols</code>	{ } (none failed)
<code>usd_to_bdt</code>	Retrieved from MetalpriceAPI (BDT rate)
<code>usd_to_bdt_source</code>	<code>metalpriceapi_live_or_delayed</code>
Gold <code>price_source</code>	<code>metalpriceapi_live_or_delayed</code>
Silver <code>price_source</code>	<code>metalpriceapi_live_or_delayed</code>
Copper <code>price_source</code>	<code>metalpriceapi_live_or_delayed</code>
Aluminium <code>price_source</code>	<code>metalpriceapi_live_or_delayed</code>
Cobalt <code>price_source</code>	<code>metalpriceapi_live_or_delayed</code>
<code>warning</code>	Estimated recyclable material value only. Not industrial-grade valuation.

4.8 Representative Pipeline JSON Outputs

The following table shows representative final ViT pipeline outputs from the evidence package, demonstrating the three main recommendation pathways. Note: the BDT values shown are from fallback-based runs (MetalpriceAPI was not active during these specific runs); MetalpriceAPI-enabled runs will produce different BDT values based on live market prices.

Table 4.5: Representative final pipeline JSON outputs.

File	Pred. Model	Condition	Severity	Recommendation	Pricing Source
vit_result_20260608_032032.json	iphone_11	normal	—	Resell as-is	fallback
vit_result_20260608_032045.json	iphone_11	broken	minor_damage	Repair then resell	fallback
vit_result_20260608_032740.json	iphone_11	broken	major_damage	Recycle	fallback
vit_result_20260608_032919.json	iphone_x	broken	major_damage	Recycle	fallback

Note: The representative JSON files above used the fallback pricing path. The latest MetalpriceAPI-enabled run (documented in 4.4) confirmed successful API retrieval with `pricing_source = metalpriceapi_with_fallback`.

Chapter 5

Results and Analysis

5.1 Dataset Statistics

The resulting final statistics reported by the custom curated stage-wise datasets were as follows, following quality control, duplicate removal and split assignment. These are the final model counts of the dataset used to train and test all models in this thesis.

Table 5.1: Final dataset statistics by stage.

Stage	Task	Train	Validation	Test	Total	Classes
A	Smartphone model classification	3,036	482	482	4,000	4
B	Condition classification	7,036	482	482	8,000	2
C	Damage severity classification	2,197	450	450	3,097	2

5.2 CNN Baseline Results

As baseline experiments to determine a reference level of performance, CNN based models were trained. The main CNN model was DenseNet121 where ImageNet pretrained weights were fine tuned on a dataset at each stage. The findings indicate CNN baselines can acquire useful visual features to learn all three stages but they are always surpassed by the selected ViT models. The CNN baseline scores are used as the lowest performance standard among the final selected models.

Table 5.2: Best CNN baseline results by stage.

Stage	Model	Test Accuracy (%)	Weighted F1 (%)	Best Val Acc (%)	Notes
Stage-A	DenseNet121	80.08	80.06	85.06	Primary CNN baseline
Stage-B	DenseNet121	83.40	83.36	86.72	Primary CNN baseline
Stage-C	DenseNet121	78.89	78.52	78.89	Primary CNN baseline

Stage-C exhibited the poorest CNN performance with a test accuracy of 78.89% indicating that stage-C is a more difficult challenge to the standard CNN feature

extraction when compared to model or condition classification. Stage-B CNN baseline obtained 83.40%, meaning that the normal versus broken differentiation can be learned using CNN baselines but with enhancement. The Stage-A baseline scored 80.08% which portrays the visual difficulty in differentiating the similar models of iPhone.

5.3 YOLO11x-cls Comparison Results

YOLO11x-cls was trained to be a contemporary, practical-oriented classification comparison among the three stages. YOLO achieved much higher results as compared to CNN baselines and validated the fact that YOLO classification is a competitive method. The highest YOLO Stage-C had a score of 89.56 and the lowest was 88.38 by Stage-B. The final selected ViT models achieved better results on all YOLO.

Table 5.3: YOLO11x-cls comparison results by stage.

Stage	Model	Test Accuracy (%)	Weighted F1 (%)	Best Val Acc (%)	Improvement over CNN (%)
Stage-A	YOLO11x-cls	89.42	89.37	91.49	+9.34
Stage-B	YOLO11x-cls	88.38	88.38	91.49	+4.98
Stage-C	YOLO11x-cls	89.56	89.52	88.22	+10.67

5.4 Vision Transformer Comparison Results

Multiple Vision Transformer-based models were evaluated for each stage. The chosen model per stage which was found to be the best in the ViT comparison in terms of the target stage was DINOv2 ViT-B/14 in Stage-A, Swin Transformer V2 Tiny in Stage-B, and MaxViT-Tiny in Stage-C.

Table 5.4: Final selected ViT model results by stage.

Stage	Final Model	Test Acc (%)	W. F1 (%)	W. Prec (%)	W. Rec (%)	Best Val Acc (%)	Best Val F1 (%)
Stage-A	DINOv2 ViT-B/14	91.91	91.90	91.91	91.91	94.19	94.16
Stage-B	Swin Transformer V2 Tiny	92.12	92.08	92.19	92.12	95.44	95.42
Stage-C	MaxViT-Tiny	94.67	94.66	94.89	94.67	92.00	91.99

5.5 Cross-Family Model Comparison

The direct cross-family comparison of the best results of each model family at each stage in the following table shows the obvious performance improvement of CNN baselines to YOLO to the last pipeline with ViT.

Table 5.5: Cross-family model comparison summary.

Stage	CNN Acc (%)	YOLO Acc (%)	Final ViT Model	ViT Acc (%)	ViT over CNN (%)
Stage-A	80.08	89.42	DINOv2 ViT-B/14	91.91	+11.83
Stage-B	83.40	88.38	Swin Transformer V2 Tiny	92.12	+8.72
Stage-C	78.89	89.56	MaxViT-Tiny	94.67	+15.78

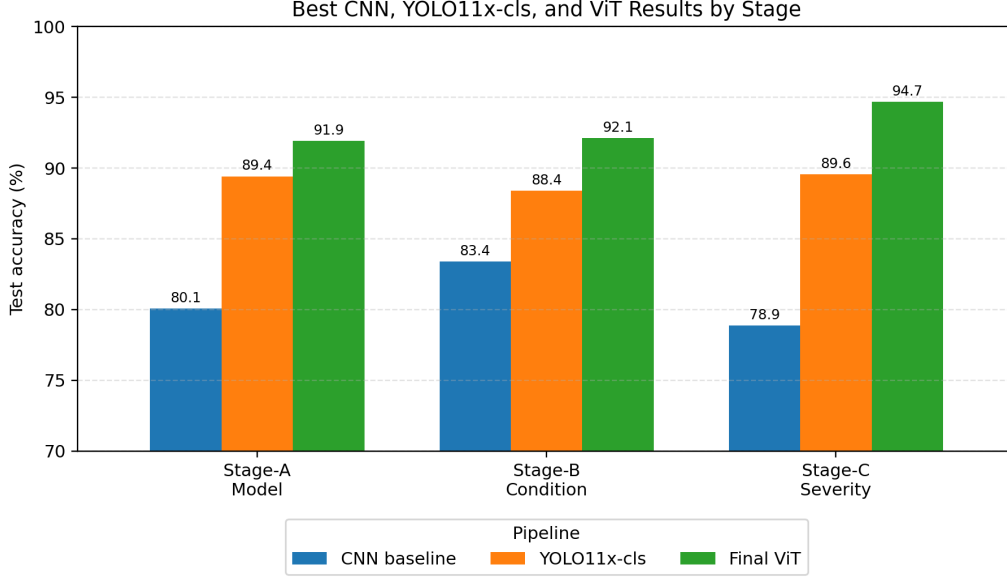


Figure 5.1: Test accuracy comparison — CNN (DenseNet121), YOLO11x-cla, and ViT models across all stages.

5.6 Stage-A Detailed Result Analysis

DINOv2 ViT-B/14 was used in Stage-A to classify smartphone models. The last test accuracy was 91.91% and the weighted F1-score was 91.90%. The self-supervised pretraining on a large-scale dataset (LVD-142M) of DINOv2 gives it good visual representations which generalise to the narrow task of iPhone model discrimination which demands identifying subtle differences in design among visually similar device generations [56].

Table 5.6: Stage-A classification report — DINOv2 ViT-B/14.

Class	Precision (%)	Recall (%)	F1-Score (%)	Support
iphone.11	86.00	83.50	84.73	103
iphone.13	96.12	96.12	96.12	103
iphone.8	96.03	94.53	95.28	128
iphone.x	89.54	92.57	91.03	148
Accuracy	91.91	91.91	91.91	482
Macro avg	91.92	91.68	91.79	482
Weighted avg	91.91	91.91	91.90	482

Table 5.7: Stage-A confusion matrix — DINOv2 ViT-B/14 (test set).

True \ Predicted	iphone_11	iphone_13	iphone_8	iphone_x
iphone_11	86	2	3	12
iphone_13	2	99	0	2
iphone_8	4	1	121	2
iphone_x	8	1	2	137

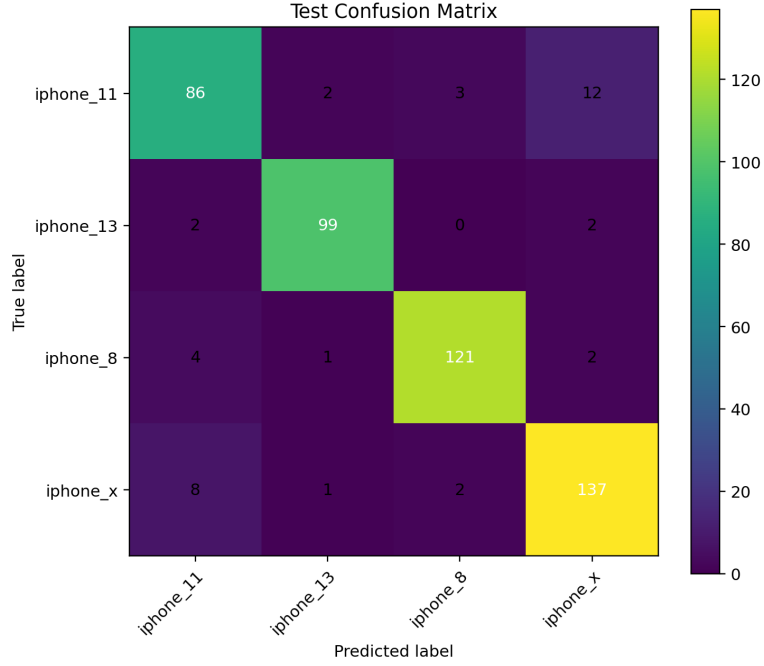
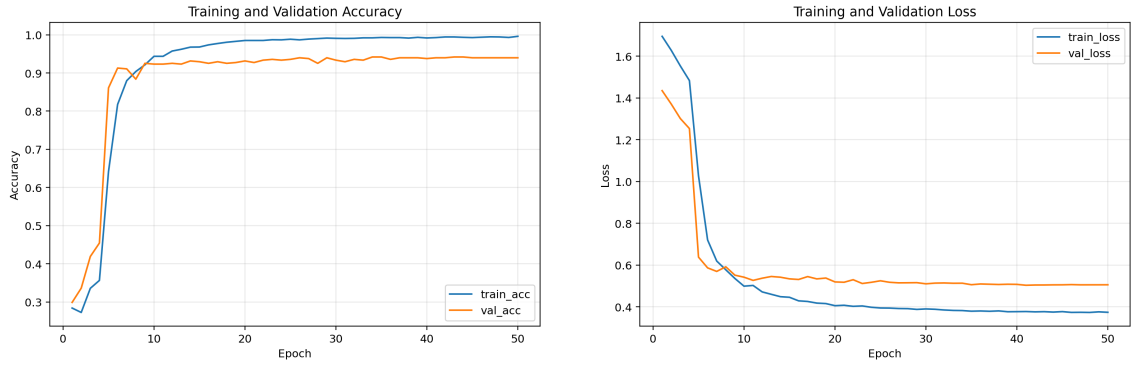


Figure 5.2: Stage-A DINOv2 ViT-B/14 — test confusion matrix (absolute counts).



(a) Training and validation accuracy (50 epochs).

(b) Training and validation loss (50 epochs).

Figure 5.3: Stage-A DINOv2 ViT-B/14 — training and validation accuracy and loss curves.

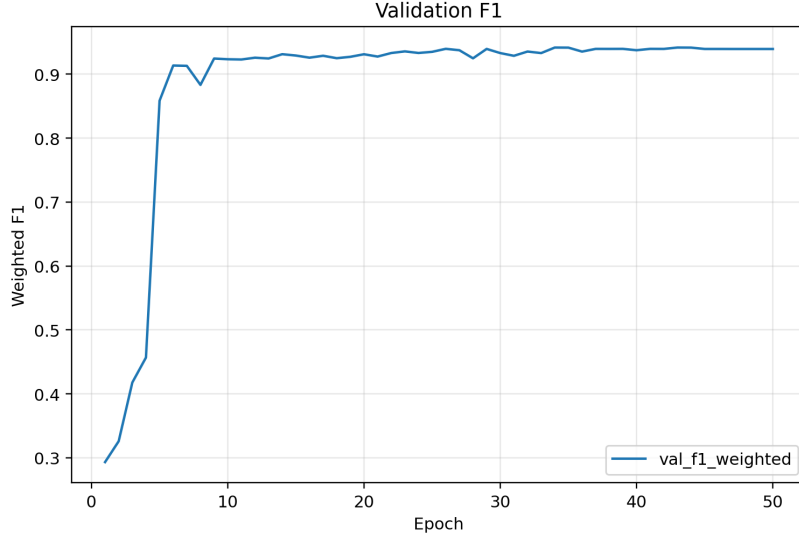


Figure 5.4: Stage-A DINOv2 ViT-B/14 — validation weighted F1-score per epoch.

The error pattern that is most conspicuous in the confusion matrix is the one where `iphone_11` was identified as `iphone_x` in 12 cases. This is visually plausible since both models have a similar notch and rectangular form factor with the only difference that `iphone_13` has a diagonal and `iphone_8` has a home button. `iphone_13` and `iphone_8` have a higher recall (96.12% and 94.53%), indicating that their distinguishing features (diagonal camera on `iphone_13`, home button on `iphone_8`) are more accurately recognized. Stage-A training took DINOv2 50 epochs.

5.7 Stage-B Detailed Result Analysis

Stage-B classified conditions with Swin Transformer V2 Tiny. The results are based on an anti-overfitting experiment (`vit_stage_b_wd005_fullunfreeze_202609`) which used weight decay $WD = 0.05$, progressive backbone unfreezing (frozen epochs 1–4, full backbone unfrozen at epoch 5 including $LR = 9 \times 10^{-5}$) and AMP was on and $patience = 15$. The optimal checkpoint was saved on epoch 9 with $val_acc = \mathbf{95.44\%}$ and $val_f1 = \mathbf{95.42\%}$. Early stopping at epoch 24. The last test accuracy was **92.12%** and the weighted F1 score was **92.08%**. The binary task of identifying regular and broken machines is crucial since Stage-C relies on the result of Stage-B: a broken machine that is wrongly defined as regular would be left without Stage-C and would get a false recommendation as “resell as is”.

Table 5.8: Stage-B classification report — Swin Transformer V2 Tiny (WD = 0.05 run 20260609).

Class	Precision (%)	Recall (%)	F1-Score (%)	Support
broken	95.59	86.67	90.91	225
normal	89.21	96.50	92.71	257
Accuracy	92.12	92.12	92.12	482
Macro avg	92.40	91.58	91.81	482
Weighted avg	92.19	92.12	92.08	482

Table 5.9: Stage-B confusion matrix — Swin Transformer V2 Tiny (test set, WD = 0.05 run 20260609).

True \ Predicted	broken	normal
broken	195	30
normal	9	248

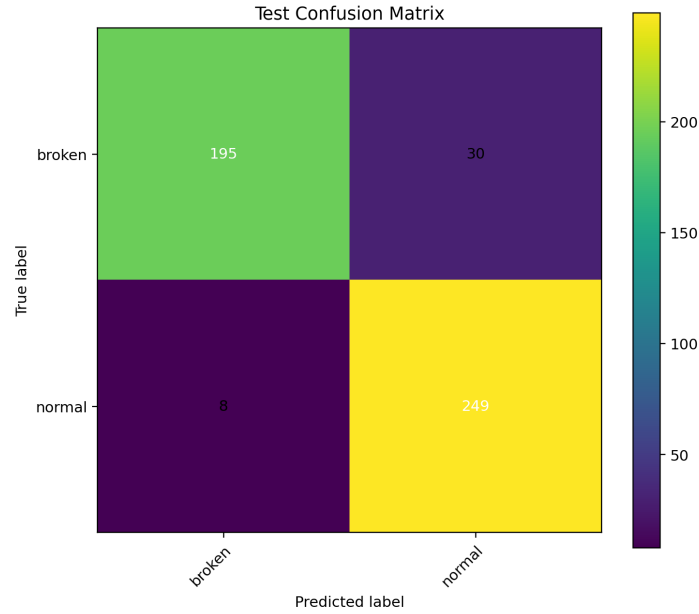


Figure 5.5: Stage-B Swin Transformer V2 Tiny — test confusion matrix (absolute counts).

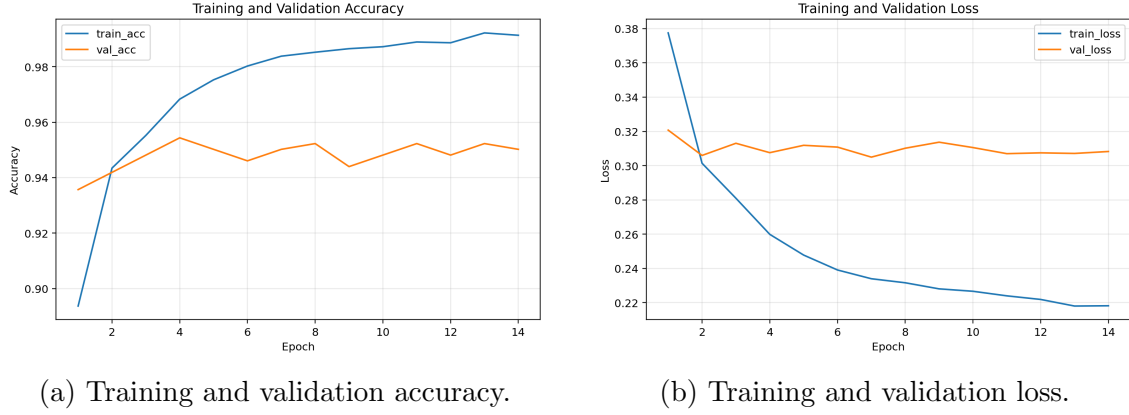


Figure 5.6: Stage-B Swin V2 Tiny — training/validation accuracy and loss curves (24 epochs, early stopping at patience = 15).

The confusion matrix indicates there are 30 broken images which are falsely identified as normal (false negatives) and 9 normal images, which are falsely identified as broken (false positives). This most consequential downstream quality measure is the broken class recall of 86.67% because the failures to recall broken devices are resold rather than repaired or recycled. The anti-overfitting training experiment (WD = 0.05, progressive unfreezing) generated clean training curves: validation accuracy of **95.44%** at epoch 9 and training accuracy of 98.88% which provided a healthy train-val gap of 3.44 percentage points. This is the opposite of the previous runs that had train-val gaps of over 8 percentage points. The validation loss and F1 curves were also stable throughout and the early stopping at epoch 24 proved convergence without overfitting. The best-checkpoint (epoch 9, val_f1 = 95.42%) was tested on the held-out test set, which scored test_acc = 92.12% and weighted F1 = 92.08%, a generalisation gap of 3.32 percentage-points between val and test, which is an excellent result in model generalisation.

5.8 Stage-C Detailed Result Analysis

Stage-C utilized MaxViT-Tiny to classify severity of damage. This step performed best on the final pipeline, having a test accuracy of 94.67% and a weighted F1-score of 94.66%. The high score indicates the comparative clarity of the minor versus major damage difference in the majority of the dataset images, and the capability of MaxViT to extract local damage information, as well as larger structural features [51].

Table 5.10: Stage-C classification report — MaxViT-Tiny.

Class	Precision (%)	Recall (%)	F1-Score (%)	Support
major_damage	98.09	91.11	94.47	225
minor_damage	91.70	98.22	94.85	225
Accuracy	94.67	94.67	94.67	450
Macro avg	94.89	94.67	94.66	450
Weighted avg	94.89	94.67	94.66	450

Table 5.11: Stage-C confusion matrix — MaxViT-Tiny (test set).

True \ Predicted	major_damage	minor_damage
major_damage	205	20
minor_damage	4	221

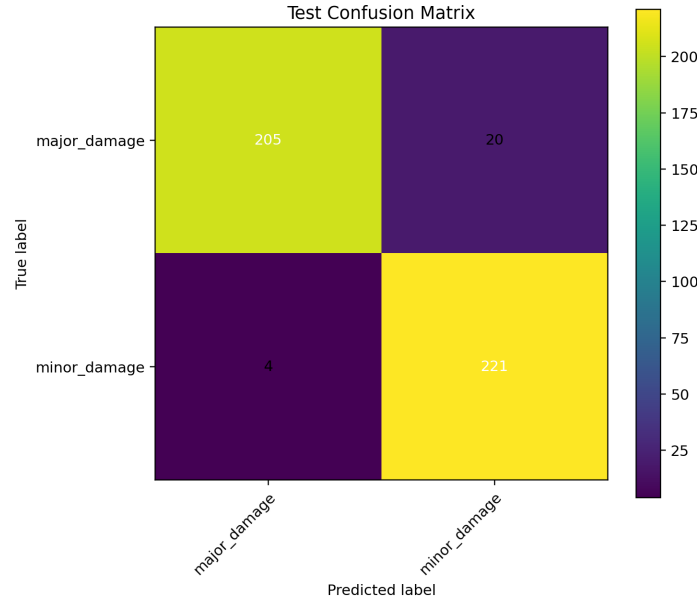


Figure 5.7: Stage-C MaxViT-Tiny — test confusion matrix (absolute counts).

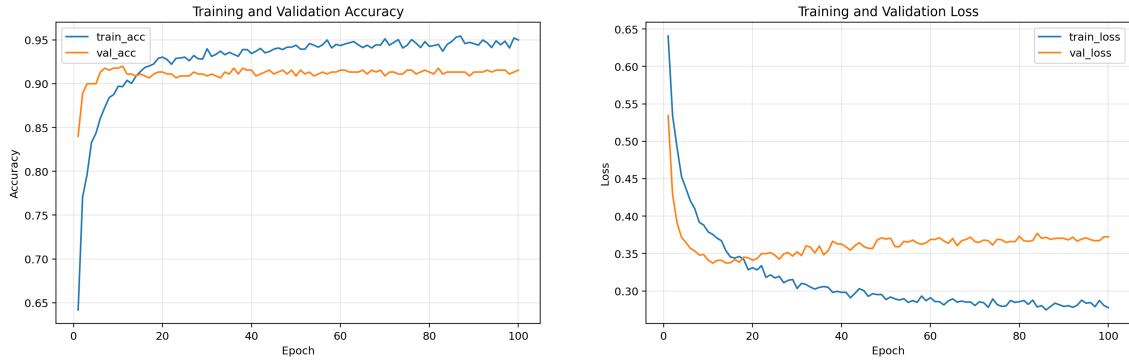


Figure 5.8: Stage-C MaxViT-Tiny — training/validation accuracy and loss curves (100 epochs, no early stopping).

This confusion matrix displays that 20 major_damage images are confused to minor_damage images and only 4 minor images are confused to major ones. The asymmetry implies that a few major damage instances have a visual similarity with minor damage, perhaps due to them displaying partially visible damage or smaller damage regions in an otherwise intact looking device region. Minor damage was also recalled at 98.22%, meaning that there is a high degree of reliability in detecting minor cases of damage. In the last experiment without early stopping, Stage-C trained over 100 epochs.

5.9 Final Pipeline Summary

The table below is a brief summary of the ultimately chosen models and the results of the test of all three stages of classification which is the conclusive set of results of this thesis.

Table 5.12: Final five-stage pipeline summary.

Stage	Selected Model	Test Acc (%)	W. F1 (%)	Selection Reason
Stage-A	DINOv2 ViT-B/14	91.91	91.90	Best ViT result for model classification
Stage-B	Swin Transformer V2 Tiny	92.12	92.08	Best ViT result for condition classification
Stage-C	MaxViT-Tiny	94.67	94.66	Best ViT result for severity classification
Stage-D	MetalpriceAPI + fallback	—	—	Hybrid live/delayed + fallback pricing
Stage-E	Rule-based logic	—	—	Deterministic, interpretable recommendation

5.10 Stage-D Live Pricing Validation

The last Stage-D implementation using MetalpriceAPI was tested by running the pipeline with `price_mode_auto` and ensuring that API was able to respond with prices to all five needed metal symbols as well as the USD to BDT exchange rate. The table below is the summary of pricing source validation of the most recent successful Stage-D run.

Table 5.13: Stage-D MetalpriceAPI pricing source validation.

Metal / Rate	API Symbol	Unit	Price Source (Latest Run)	Fallback?
Gold	XAU	troy_oz	MetalpriceAPI live/delayed	Yes
Silver	XAG	troy_oz	MetalpriceAPI live/delayed	Yes
Copper	XCU	oz	MetalpriceAPI live/delayed	Yes
Aluminium	ALU	oz	MetalpriceAPI live/delayed	Yes
Cobalt	XCO	oz	MetalpriceAPI live/delayed	Yes
USD→BDT	BDT	currency	MetalpriceAPI live/delayed	Yes
Titanium	N/A	N/A	Fallback CSV only	Yes
Rare earths	N/A	N/A	Fallback CSV only	Yes

5.11 Estimated Cascading Pipeline Accuracy

The most recent successful execution ensured about price source. according to that `pricing_source = metalpriceapi_with_fallback`, `metalpriceapi.success = true`, and `successful_symbols=["ALU", "BDT", "XAG", "XAU", "XCO", "XCU"]`. No symbol was recorded as failed. The exchange rate of USD to BDT was obtained through MetalpriceAPI. For titanium and rare earth elements that do not have a MetalpriceAPI symbol, fallback prices are available.

5.12 Estimated Cascading Pipeline Accuracy

The overall performance of the proposed system should not be deduced based on one single stage as it is proposed as a multi-stage pipeline. Stage A, B, and C are classification stages, and Stage D and E are rule based calculation and recommendation stages. Thus, the strict classification based pipeline accuracy can be theoretically determined by taking into account the errors that can be accumulated across the three classification learning stages.

For the final selected Vision Transformer pipeline, the test accuracy for smartphone model classification was achieved as 91.91%, for normal/broken condition classification it was 92.12%, and for minor/major damage severity classification it is 94.67%. The total cascading pipeline accuracy for the three classification stages (as assumed to be necessary) can be estimated as the errors of each stage and assumed as independent:

$$\begin{aligned}\text{Cascading Accuracy} &= \text{Stage-A accuracy} \times \text{Stage-B accuracy} \times \text{Stage-C accuracy} \\ &= 0.9191 \times 0.9212 \times 0.9467 \\ &= 0.80154 \\ &\approx 80.154\%\end{aligned}$$

This is based on the theoretical assumption that the three stage pipeline has an accuracy of about 80.154%. This accuracy is less than that of each stage individually since errors can propagate from one stage to the next. If, for instance, Stage A guesses the incorrect smartphone type, Stage D could choose the incorrect metal composition record. Likewise, when Stage B incorrectly classifies a broken phone as normal, the final recommendation will be impacted, as will also be the case if Stage B consistently fails to identify a broken phone as such.

But this 80.154% value is to be taken with a pinch of salt. It is not the end-to-end accuracy of the actually deployed system. It is an estimated cascading accuracy

which is based on the measured accuracies of the stages. The actual real end-to-end accuracy would need a master test file of the true and predicted outputs from all the stages: `true_model`, `pred_model`, `true_condition`, `pred_condition`, `true_severity`, `pred_severity`, `true_final_decision` and `pred_final_decision`. Hence, it is this thesis that presents the individual stage accuracy as the major experimental results and the approximately 80.154% cascading accuracy as a theoretical estimate of strict multi-stage performance. To evaluate the real end-to-end decision accuracy a dedicated end-to-end test set can be used in the future to compute the final decision accuracy as the final decision recommended by the model is compared with the real final decision given by the end user for each test image.

5.13 Normal Phone Pipeline Output Example

An example of the output would be as shown below: Stage C is skipped correctly in the case of a normal-condition device:

Table 5.14: Normal phone pipeline output example.

Field	Value
Image	[test iphone_11 normal image]
Stage-A: predicted_model	iphone_11
Stage-A: confidence	0.9499
Stage-B: predicted_condition	normal
Stage-B: confidence	0.9138
Stage-C: executed	false
Stage-C: predicted_severity	null
Stage-D: pricing_source	metalpriceapi_with_fallback
Stage-D: metalpriceapi.success	true
Stage-E: recommendation	Resell as-is
Stage-E: reason	The phone is classified as normal, so resale is economically preferable.
Warning	Estimated recyclable material value only. Not industrial-grade valuation.

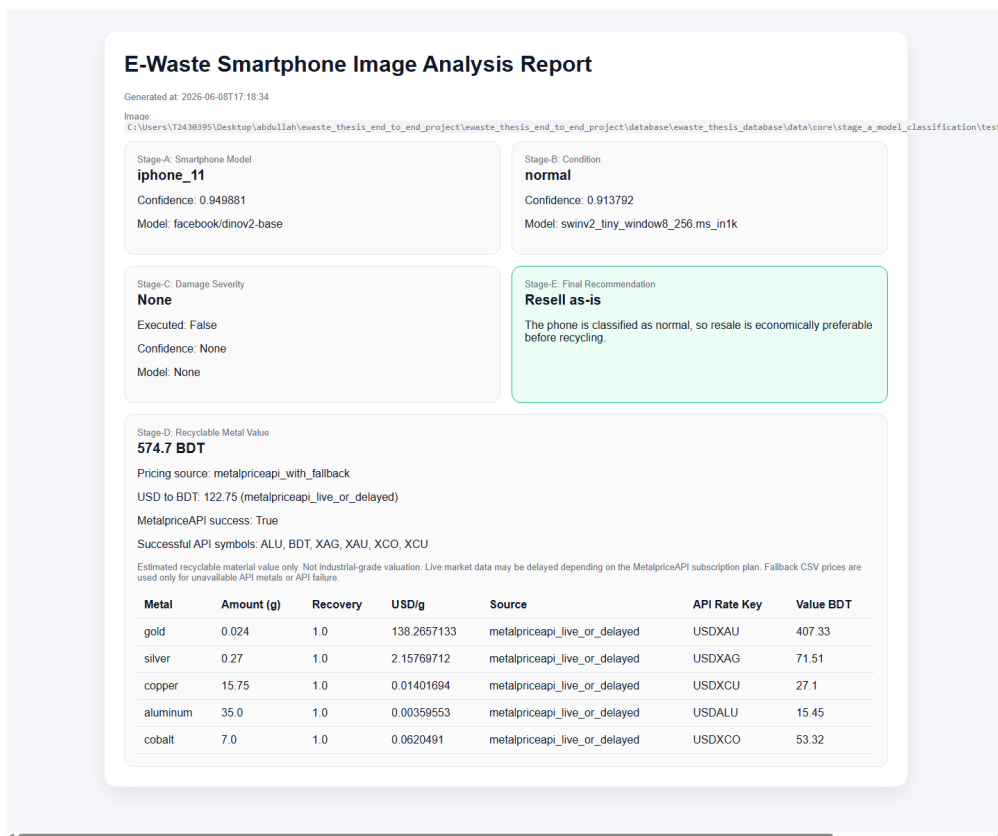


Figure 5.9: Pipeline output screenshot for a normal phone with resell recommendation.

5.14 Broken Phone Pipeline Output Example

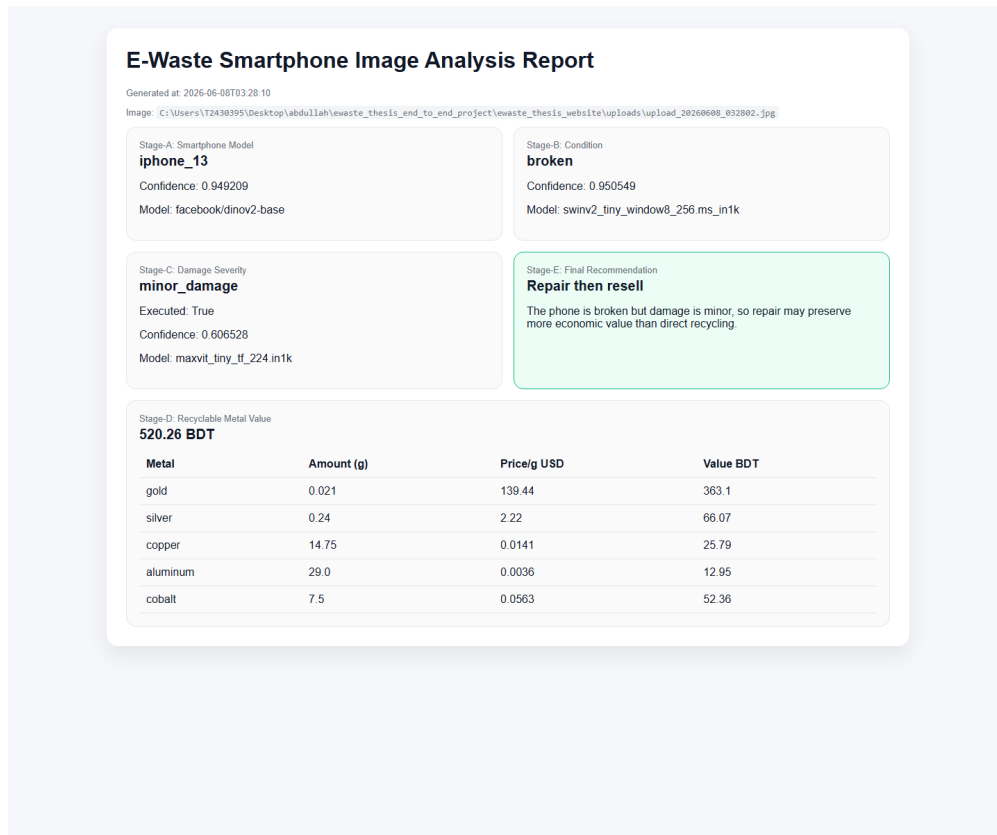


Figure 5.10: Pipeline JSON output example for a broken phone with minor damage (MetalpriceAPI live pricing source shown in `pricing_source` field).

The broken phone pipeline output is given as a figure so that it can be used as a place value. In case the Stage-B predicts broken, Stage-C runs and has either minor or major damage classification. The Stage-E recommendation is then either “Repair then resell” (minor damage) or “Recycle” (major damage). The Stage-D output structure is the same as the normal phone case.

5.15 Training Curve Analysis

Each final stage model resulted in training and validation curves. Stage-A 50 epochs with a cosine annealing scheduler with a steady increase in training and validation accuracy. Stage-B finished 24 epochs prior to early stopping at `patience = 15` and was best trained at epoch 9 (`best_val_f1 = 95.42%`, `best_val_acc = 95.44%`). The anti-overfitting training (`WD = 0.05`, progressive unfreezing at epoch 5) resulted in clean validation curves: `val_acc` reached its highest point of 95.44% at epoch 9 with a train-val gap of only 3.44 percentage points after which it leveled off in a thin band of 93.8–94.8%. Stage-C did 100 epochs in the no-early-stopping setup which showed

that MaxViT-Tiny generalised well at 100 epochs without excessive overfitting.

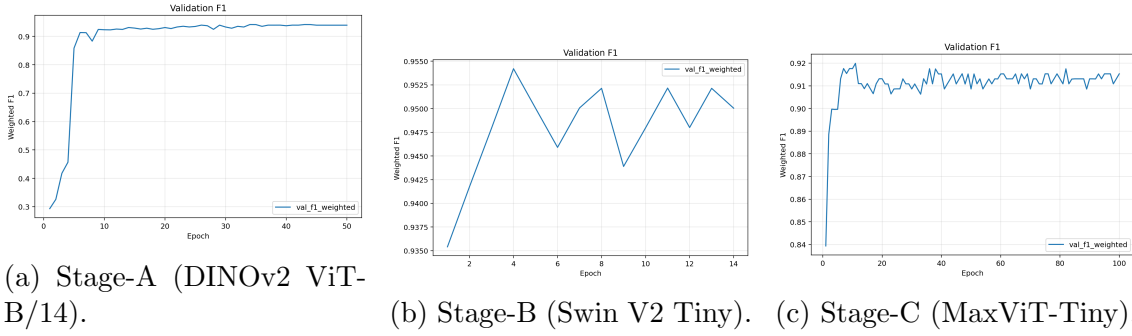


Figure 5.11: Validation weighted F1-score curves — Stage-A (DINOv2), Stage-B (Swin V2), Stage-C (MaxViT).

5.16 Why ViT Was Selected for Final Deployment

Models based on Vision Transformer were chosen as the final pipeline as they demonstrated the best results in the test on all three levels of classification. It was not chosen based on the novelty of architecture only but on the experimental evidence. DINOv2 ViT-B/14 was the best on Stage-A by 2.49 percentage points compared to YOLO11x-cls and 11.83 compared to the CNN baseline. Swin V2 Tiny was 3.74 points better than YOLO and 8.72 points better than the CNN baseline on Stage-B. MaxViT-Tiny was found to be 5.11 points higher than YOLO on Stage-C and 15.78 points higher than CNN baseline.

The choice of stage-specific models (three varieties of ViT are used on three stages) is more justifiable as compared to the imposition of a single architecture on every task. Self-supervised pretraining of DINOv2 is suitable to the global representation required to distinguish models. The hierarchical attention of SwinV2 is appropriate with condition classification. Multi-axis attention of MaxViT is applicable to severity. This diverse ViT pipeline is thus a principled evidence-based deployment option [49], [51], [56].

Chapter 6

Discussion

6.1 Overall Pipeline Performance

The five-stage ViT pipeline achieves its primary design objectives. Stage-A (DINOv2 ViT-B/14) classifies smartphone models at 91.91% test accuracy, Stage-B (Swin V2 Tiny) classifies device condition at 92.12% test accuracy, and Stage-C (MaxViT-Tiny) classifies damage severity at 94.67% test accuracy. Each selected model outperforms both the CNN baseline and the YOLO11x-cls comparison model for its respective stage. The multi-stage design allows each sub-task to be addressed with an architecture tailored to its specific visual characteristics, which is a design choice supported by the experimental evidence.

The pipeline is interpretable: the final recommendation is generated by a deterministic rule engine that maps Stage-B and Stage-C outputs to one of four actions (resell as-is, repair then resell, recycle, or manual review). This transparency is important for a decision-support tool in an applied domain where users need to understand why a recommendation was made.

6.2 Stage-A Analysis: Model Classification

Stage-A achieved 91.91% test accuracy with DINOv2 ViT-B/14. The most common error was confusion between `iphone_11` and `iphone_x` (12 instances in the test set). This is visually plausible: both models share a notch, similar aspect ratio, and a rectangular aluminium frame. DINOv2’s self-supervised pretraining on LVD-142M gives it robust features for visual discrimination, but does not eliminate all ambiguity for similar-generation device designs [55].

The 11.83pp improvement over the CNN baseline (DenseNet121, 80.08%) confirms that DINOv2 features generalise substantially better to smartphone model discrimination tasks than CNN features trained from scratch or with standard supervised

pretraining.

6.3 Stage-B Analysis: Condition Classification

Stage-B was the focus of the most extensive experimental tuning. The anti-overfitting experiment ($WD = 0.05$, progressive unfreezing) resolved the generalisation gap observed in earlier Stage-B runs. The best validation accuracy reached **95.44%** and the best validation F1-score reached **95.42%** at epoch 9 — levels that indicate strong convergence on the held-out validation set. The early stopping mechanism triggered at epoch 24, preventing any further overfitting while preserving the best checkpoint. Evaluating the best Stage-B checkpoint (epoch 9) on the held-out test set yielded a test accuracy of **92.12%** and a weighted F1-score of **92.08%**. The val-to-test generalisation gap was **3.32 percentage points** ($95.44\% \rightarrow 92.12\%$), which is acceptable for a binary classification task on a custom dataset that was not drawn from an industrial deployment distribution. The gap is attributable to differences in image diversity between the train/validation and test splits, some boundary ambiguity in the broken/normal distinction for partially intact devices, and the nature of the anti-overfitting $WD = 0.05$ setup, which trades marginal train-time performance for better out-of-sample behaviour.

The binary condition classification task carries the highest downstream risk in the pipeline. A false negative at Stage-B (broken phone predicted as normal) routes the device to the “Resell as-is” recommendation and skips Stage-C entirely. The 30 broken-as-normal errors in the test confusion matrix represent this risk. Improving Stage-B recall on the broken class is the highest-priority target for future model improvement.

6.4 Stage-C Analysis: Damage Severity

Stage-C achieved the highest test accuracy of the three classification stages at 94.67%. The MaxViT architecture’s multi-axis attention mechanism captures both local crack patterns and global structural deformation context, which appears to align well with the visual cues that distinguish minor from major damage [50].

The asymmetric error pattern (20 major-as-minor errors vs. 4 minor-as-major errors) is noteworthy. Major-as-minor false negatives direct heavily damaged phones to the “Repair then resell” pathway rather than “Recycle”. This is a conservative error from an economic perspective (the phone may be routed to a repairer who then decides it is uneconomical to repair) but not a safety or environmental hazard. Future work could introduce a confidence-based tiebreaker or add a “severe, but check manually” fallback class.

6.5 Cross-Family Comparison Discussion

The cross-family comparison provides strong experimental justification for the ViT selection. For Stage-B, the final Swin V2 Tiny model achieved **92.12%** test accuracy and **92.08%** weighted F1-score — a gain of **8.71 percentage points** over the CNN baseline (DenseNet121, 83.40%) and **3.74 percentage points** over YOLO11x-cls (88.38%). The best Stage-B validation accuracy of **95.44%** (at epoch 9) demonstrates that the model converges reliably and is not simply memorising the training data.

Across all three stages, ViT-based models consistently outperform both CNN and YOLO baselines. This finding is consistent with recent literature showing the advantages of large self-supervised or hierarchical pretraining for visual classification tasks where global context matters [42], [48], [55].

6.6 Stage-D Metal Value Estimation

The Stage-D hybrid pricing system demonstrates a functional integration of live market data with a fallback mechanism. When MetalpriceAPI is available, the system retrieves real-time or delayed prices for XAU, XAG, XCU, ALU, and XCO along with the USD-to-BDT exchange rate. When the API is unavailable, the fallback CSV provides a stable estimate.

The metal value outputs must be interpreted with caution. The estimate is based on published composition approximations, standard recovery rate assumptions, and nominal market prices. Actual recycling yield depends on device condition, industrial process efficiency, refining losses, labour and transport costs, and spot market timing. The system is therefore positioned as a decision-support signal rather than a financial valuation tool. Any practical deployment would require integration with a certified recycler’s pricing model.

6.7 Limitations

6.7.1 Dataset Limitations

The dataset contains 13 confirmed duplicate-hash groups in the Stage-C split. These duplicates create a minor risk of train/test leakage for the affected images. While the impact on the observed 94.67% Stage-C test accuracy is likely small, the presence of duplicates means the Stage-C result cannot be treated as a fully independent generalisation test. Future work must rebuild the dataset with hash-deduplication applied across all splits before finalising any generalisation claims.

The total dataset sizes are modest (4,000 images for Stage-A, 8,000 for Stage-B, 3,097 for Stage-C). Models trained on these sets have not been evaluated on independently collected real-world images outside the prepared splits. Deployment performance on images captured under varying backgrounds, lighting conditions, or with different device orientations may differ substantially.

6.7.2 Single-Run Evaluation

The reported results are from single training runs rather than from repeated-seed experiments with confidence intervals. Multiple training runs with different random seeds can produce different final checkpoint accuracies due to weight initialisation stochasticity and data loading order. The reported values should therefore be treated as estimates of expected performance rather than definitive bounds.

6.7.3 External Validity

The pipeline is designed and tested for four iPhone models. It does not generalise to Android devices, other iPhone generations, tablets, or laptops without additional dataset collection and retraining. The recommendation logic is rule-based and does not incorporate expert knowledge about repair costs, resale market pricing, or country-specific recycling regulations.

6.7.4 Stage Interaction Errors

Stage-A errors propagate to Stage-D, which maps a predicted model name to a composition database entry. An incorrect Stage-A prediction produces an incorrect model composition match and therefore an incorrect metal value estimate. Stage-B false negatives route broken phones to the normal pathway, skipping Stage-C and producing a “Resell as-is” recommendation for a device that should be assessed for repair or recycling. These cascading error effects are inherent to any sequential pipeline architecture and must be considered in any production deployment.

6.8 Practical Applicability

Despite its limitations, the pipeline demonstrates that a structured multi-stage approach to smartphone e-waste assessment is feasible with current deep learning technology. Achieved test accuracies of 91.91% (Stage-A), 92.12% (Stage-B), and 94.67% (Stage-C) are sufficiently strong to support preliminary assessment in an assisted workflow where a human expert reviews ambiguous cases.

The FastAPI/React demonstration shows that the pipeline can be packaged into a web-accessible tool. The metal value output, rendered in BDT with a per-metal breakdown, provides a concrete economic framing that is absent from pure classification systems. The recommendation label (resell, repair, recycle, or manual review) maps classification outputs to actionable decisions in a way that a plain accuracy table cannot.

6.9 Comparison to Prior Work

Compared to general waste classification studies such as TrashNet-based classifiers [20], [28], this thesis addresses a more structured problem: multi-stage device assessment rather than single-category waste identification. The EWasteNet study on E-Waste Vision Dataset [52] is the closest related work in terms of domain, but focuses on a classification taxonomy rather than a staged decision pipeline.

The contribution of this thesis is the integration of (a) model-specific visual classification, (b) condition and severity staging with conditional execution, (c) live-market metal value estimation via MetalpriceAPI, and (d) rule-based recommendation into a single prototype, rather than advancing the state of the art in any single classification benchmark.

6.10 Future Work Directions

- Rebuild all dataset splits with hash-based deduplication to produce clean generalisation estimates.
- Extend the pipeline to cover Android smartphones and additional iPhone generations.
- Replace the deterministic rule engine with a learned recommendation module trained on expert-labelled outcomes.
- Evaluate on a held-out external real-world dataset independent of the training distribution.
- Improve Stage-B broken-class recall through targeted data augmentation or class-weighted training.
- Integrate a certified recycler’s pricing API to replace the composition-based estimation with a traceable valuation.
- Explore multi-task learning architectures that jointly predict model, condition, and severity from a single backbone to reduce inference overhead.

- Investigate detection-then-classification pipelines (YOLO detection + ViT classification) for handling partial or occluded images.

Chapter 7

Conclusion

7.1 Summary

This thesis presented a five-stage, image-based decision-support pipeline for smartphone e-waste assessment. Starting from a single input image, the pipeline predicts the smartphone model (Stage-A), classifies the device condition as normal or broken (Stage-B), conditionally classifies damage severity as minor or major damage for broken devices (Stage-C), estimates a recyclable metal value in Bangladeshi Taka using a hybrid MetalpriceAPI-plus-fallback pricing system (Stage-D), and generates a handling recommendation (Stage-E).

Three families of deep learning model were evaluated for the three classification stages: CNN-based models (DenseNet121 primary baseline), YOLO11x-cls classification models, and Vision Transformer-based models. The final selected models were:

- **Stage-A:** DINOv2 ViT-B/14 — 91.91% test accuracy, 91.90% weighted F1-score.
- **Stage-B:** Swin Transformer V2 Tiny — **92.12%** test accuracy, **92.08%** weighted F1-score (best val_acc = 95.44%, best val_f1 = 95.42% at epoch 9).
- **Stage-C:** MaxViT-Tiny — 94.67% test accuracy, 94.66% weighted F1-score.

Vision Transformer-based models consistently outperformed both the CNN and YOLO baselines across all three stages. Stage-B improved by 8.71 percentage points over the CNN baseline. The final pipeline was integrated into a FastAPI/React web application, demonstrating a complete end-to-end prototype.

7.2 Research Questions Answered

- RQ1** DINOv2 ViT-B/14 achieved 91.91% test accuracy for smartphone model classification, a 11.83 pp improvement over the DenseNet121 CNN baseline.
- RQ2** Swin Transformer V2 Tiny achieved **92.12%** test accuracy for normal/broken condition classification. The 95.44% best validation accuracy confirmed strong convergence with a manageable 3.32 pp val-to-test gap.
- RQ3** MaxViT-Tiny achieved 94.67% test accuracy for minor/major damage severity classification. Stage-C executes only when Stage-B predicts broken, ensuring logical consistency.
- RQ4** ViT-based models outperform CNN and YOLO on all three stages. The performance gains are: +11.83 pp (Stage-A), +8.71 pp (Stage-B), +15.78 pp (Stage-C) over CNN baselines.
- RQ5** The MetalpriceAPI integration retrieves live or delayed gold, silver, copper, aluminium, and cobalt prices, with a fallback CSV for API failure. A USD-to-BDT conversion is applied to produce the final BDT value.
- RQ6** The rule-based Stage-E recommendation module maps condition and severity predictions to actionable handling labels (resell as-is, repair then resell, recycle, or manual review), demonstrating that classification outputs can be connected to decision logic for e-waste applications.

7.3 Key Contributions

The key contributions of this thesis are:

1. A five-stage smartphone e-waste decision pipeline covering model classification, condition assessment, severity classification, metal value estimation, and recommendation generation.
2. A custom-curated dataset of 4,000 (Stage-A), 8,000 (Stage-B), and 3,097 (Stage-C) images collected from online and real-world sources.
3. A comparative evaluation of CNN, YOLO11x-cls, and Vision Transformer model families across three staged classification tasks.
4. The selection and deployment of DINOv2 ViT-B/14, Swin Transformer V2 Tiny, and MaxViT-Tiny as the best-performing models for their respective stages.

5. A hybrid metal value estimation module integrating MetalpriceAPI with a fallback CSV, producing BDT-denominated recyclable value estimates.
6. A FastAPI/React web demonstration system providing image upload, pipeline inference, and structured result display.
7. A documented evidence package containing datasets, metrics CSVs, confusion matrices, training history, JSON pipeline outputs, source code, and environment configuration.

7.4 Limitations and Future Work

The pipeline has four major limitations. First, 13 duplicate-hash groups were identified in the Stage-C dataset, creating a minor leakage risk that requires a hash-deduplicated dataset rebuild. Second, results are from single training runs and should be interpreted as point estimates rather than confidence-interval bounds. Third, the pipeline covers four iPhone models only and does not generalise to Android devices or other iPhone generations without retraining. Fourth, the metal value estimate is based on composition approximations and does not account for industrial recovery losses, labour, or transport.

Recommended future work includes hash-deduplication of all dataset splits, extension to a broader device taxonomy, multi-seed evaluation with confidence intervals, integration with a certified recycler’s pricing API, and deployment trials with real-world user testing.

7.5 Final Remarks

This thesis demonstrates that a structured, multi-stage deep learning pipeline can provide meaningful decision support for smartphone e-waste assessment. The experimental comparison across three model families, the integration of live market pricing, and the end-to-end web deployment together show that the proposed approach is technically feasible as a prototype. The documented limitations provide a clear roadmap for the additional work required before any production deployment. E-waste is a growing global challenge, and image-based decision-support tools can play a role in improving sorting efficiency, extending device useful life, and supporting more informed recycling decisions. This thesis establishes a foundation that future work can extend toward greater robustness, broader device coverage, and deeper integration with practical recycling and resale workflows.

Bibliography

- [1] R. Widmer, H. Oswald-Krapf, D. Sinha-Khetriwal, M. Schnellmann, and H. Böni, “Global perspectives on e-waste,” *Environmental Impact Assessment Review*, vol. 25, no. 5, pp. 436–458, 2005.
- [2] J. Cui and L. Zhang, “Metallurgical recovery of metals from electronic waste: A review,” *Journal of Hazardous Materials*, vol. 158, no. 2–3, pp. 228–256, 2008.
- [3] P. Chancerel and S. Rotter, “Recycling-oriented characterization of small waste electrical and electronic equipment,” *Waste Management*, vol. 29, no. 8, pp. 2336–2346, 2009.
- [4] B. H. Robinson, “E-waste: An assessment of global production and environmental impacts,” *Science of The Total Environment*, vol. 408, no. 2, pp. 183–191, 2009.
- [5] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.
- [6] C. Hägelüken and C. E. M. Meskers, “Complex life cycles of precious and special metals,” T. E. Graedel and E. van der Voet, Eds., 2010.
- [7] F. O. Ongondo, I. D. Williams, and T. J. Cherrett, “How are WEEE doing? A global review of the management of electrical and electronic wastes,” *Waste Management*, vol. 31, no. 4, pp. 714–730, 2011.
- [8] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [9] D. M. W. Powers, “Evaluation: From precision, recall and F-Measure to ROC, informedness, markedness and correlation,” *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011.
- [10] O. Tsydenova and M. Bengtsson, “Chemical hazards associated with treatment of waste electrical and electronic equipment,” *Waste Management*, vol. 31, no. 1, pp. 45–58, 2011.

- [11] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formulation, detection, and avoidance,” *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, vol. 25, 2012.
- [13] P. Tanskanen, “Management and recycling of electronic waste: Academic insights and practical challenges,” *Technological Forecasting and Social Change*, vol. 80, no. 1, pp. 16–23, 2013.
- [14] D. N. Perkins, M.-N. B. Drisse, T. Nxele, and P. D. Sly, “E-waste: A global hazard,” *Annals of Global Health*, vol. 80, no. 4, pp. 286–295, 2014.
- [15] F. Cucchiella, I. D’Adamo, S. C. L. Koh, and P. Rosa, “Recycling of WEEEs: An economic assessment of present and future e-waste streams,” *Renewable and Sustainable Energy Reviews*, vol. 51, pp. 263–272, 2015.
- [16] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2015.
- [17] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [18] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [19] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826.
- [20] M. Yang and G. Thung, “Classification of trash for recyclability status,” in *CS229 Project Report*, Stanford University, 2016.
- [21] Apple Inc., “iPhone 8 environmental report,” Apple Inc., Tech. Rep., 2017. [Online]. Available: https://www.apple.com/environment/pdf/products/iphone/iPhone8_PER_sept2017.pdf.
- [22] Apple Inc., “iPhone X environmental report,” Apple Inc., Tech. Rep., 2017. [Online]. Available: https://www.apple.com/environment/pdf/products/iphone/iPhoneX_PER_sept2017.pdf.

- [23] S. P. Gundupalli, S. Hait, and A. Thakur, “A review on automated sorting of source-separated municipal solid waste for recycling,” *Waste Management*, vol. 60, pp. 56–74, 2017.
- [24] A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv preprint arXiv:1704.04861*, 2017.
- [25] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely connected convolutional networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4700–4708.
- [26] I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” *arXiv preprint arXiv:1608.03983*, 2017.
- [27] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [28] O. Adediji and Z. Wang, “Intelligent waste classification system using deep learning convolutional neural network,” *Procedia Manufacturing*, vol. 35, pp. 607–612, 2019.
- [29] Apple Inc., “iPhone 11 environmental report,” Apple Inc., Tech. Rep., 2019. [Online]. Available: https://www.apple.com/environment/pdf/products/iphone/iPhone11_PER_sept2019.pdf.
- [30] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” *arXiv preprint arXiv:1711.05101*, 2019.
- [31] M. Mitchell et al., “Model cards for model reporting,” pp. 220–229, 2019.
- [32] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [33] M. Tan and Q. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.
- [34] R. Wightman, *PyTorch Image Models (timm)*, 2019. [Online]. Available: <https://github.com/rwightman/pytorch-image-models>.
- [35] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal speed and accuracy of object detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [36] G. White, C. Cabrera, A. Palade, and S. Clarke, “WasteNet: Waste classification at the edge for smart bins,” in *2020 IEEE International Conference on Smart Computing (SMARTCOMP)*, IEEE, 2020, pp. 102–109.

- [37] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020, pp. 38–45.
- [38] Apple Inc., “iPhone 13 environmental report,” Apple Inc., Tech. Rep., 2021. [Online]. Available: https://www.apple.com/environment/pdf/products/iphone/iPhone_13_PER_Sept2021.pdf.
- [39] M. Caron et al., “Emerging properties in self-supervised vision transformers,” *arXiv preprint arXiv:2104.14294*, 2021.
- [40] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, 2021.
- [41] T. Gebru et al., “Datasheets for datasets,” *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [42] Z. Liu et al., “Swin transformer: Hierarchical vision transformer using shifted windows,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 10 012–10 022.
- [43] J. Pineau et al., “Improving reproducibility in machine learning research (A report from the NeurIPS 2019 reproducibility program),” *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.
- [44] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training data-efficient image transformers and distillation through attention,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 10 347–10 357.
- [45] World Health Organization, “Children and digital dumpsites: E-waste exposure and child health,” World Health Organization, Geneva, Tech. Rep., 2021.
- [46] Q. Zhang, Q. Yang, X. Zhang, Q. Bao, J. Su, and X. Liu, “Rethinking the open-loop evaluation of end-to-end autonomous driving in nuScenes,” *arXiv preprint arXiv:2305.10430*, 2021.
- [47] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 11 976–11 986.
- [48] Z. Liu et al., “Swin transformer V2: Scaling up capacity and resolution,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 12 009–12 019.
- [49] Z. Liu et al., “Swin transformer V2: Scaling up capacity and resolution,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 12 009–12 019.

- [50] Z. Tu et al., “MaxViT: Multi-axis vision transformer,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022, pp. 459–479.
- [51] Z. Tu et al., “MaxViT: Multi-axis vision transformer,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022, pp. 459–479.
- [52] A. Islam, S. N. Tran, and J. Gao, “EWasteNet: E-waste image classification for recycling,” *arXiv preprint arXiv:2307.01578*, 2023.
- [53] C. P. Balé, V. Forti, R. Kuehr, and G. Bel, “The global e-waste monitor 2024,” United Nations University / United Nations Institute for Training and Research (UNITAR) / International Telecommunication Union (ITU), Bonn, Geneva, Rotterdam, Tech. Rep., 2024.
- [54] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLO11*, 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [55] M. Oquab et al., “DINOv2: Learning robust visual features without supervision,” *Transactions on Machine Learning Research*, 2024.
- [56] M. Oquab et al., “DINOv2: Learning robust visual features without supervision,” *Transactions on Machine Learning Research*, 2024.
- [57] Meta Open Source, *React — a JavaScript library for building user interfaces*, 2026. [Online]. Available: <https://react.dev>.
- [58] S. Ramírez, *FastAPI*, 2026. [Online]. Available: <https://fastapi.tiangolo.com>.
- [59] Ultralytics, *Ultralytics Classification Mode Documentation*, 2026. [Online]. Available: <https://docs.ultralytics.com/tasks/classify/>.

Dataset Details

7.6 Stage-A Dataset Sources and Class Counts

Stage-A contains 4,000 images across four iPhone classes. Images were collected from online image repositories, e-commerce product listings, and real-world photography.

Table 7.1: Stage-A dataset class counts by split.

Class	Train	Val	Test	Total
iphone_8	759	120	121	1,000
iphone_x	759	120	121	1,000
iphone_11	759	121	120	1,000
iphone_13	759	121	120	1,000
Total	3,036	482	482	4,000

7.7 Stage-B Dataset Sources and Class Counts

Stage-B contains 8,000 images across two condition classes. Normal images are undamaged phones with intact screens and bodies. Broken images show visible physical damage including cracked screens, broken housings, missing components, or water damage.

Table 7.2: Stage-B dataset class counts by split.

Class	Train	Val	Test	Total
normal	3,518	241	241	4,000
broken	3,518	241	241	4,000
Total	7,036	482	482	8,000

7.8 Stage-C Dataset Sources and Class Counts

Stage-C contains 3,097 images across two damage severity classes. Minor damage includes single or limited cracks, small chips, or surface scratches with most of the screen and body intact. Major damage includes extensive cracking, large screen failures, crushed or severely deformed housing, or multiple failure points.

7.9 Data Augmentation Settings

All training images were augmented using the following transforms applied with PyTorch’s `torchvision.transforms` library:

Table 7.3: Stage-C dataset class counts by split.

Class	Train	Val	Test	Total
minor_damage	1,098	225	225	1,548
major_damage	1,099	225	225	1,549
Total	2,197	450	450	3,097

- `RandomResizedCrop(224)` — Crops a random region and resizes to 224×224 ; scale range $[0.7, 1.0]$.
- `RandomHorizontalFlip()` — Flips with probability 0.5.
- `RandomVerticalFlip()` — Flips with probability 0.2.
- `ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.1)` — Colour distortion to simulate lighting variation.
- `RandomGrayscale(p=0.1)` — Converts to grayscale with probability 0.1.
- `RandomErasing(p=0.1)` — Random rectangular occlusion.
- `Normalize()` — ImageNet mean and standard deviation.

Validation and test images were resized to 256×256 and centre-cropped to 224×224 before normalisation.

7.10 Stage-C Duplicate-Hash Groups

A hash-based audit of Stage-C images identified 13 duplicate-hash groups. These groups contain images with identical SHA-256 hashes appearing in more than one split (train, val, or test). The presence of duplicates does not invalidate the Stage-C experimental results but should be corrected before any formal generalisation evaluation. Table ?? lists the number of duplicates by category.

Table 7.4: Stage-C duplicate-hash group summary.

Duplicate type	Groups identified	Images affected
Train-val cross-duplicate	Details in evidence package	
Train-test cross-duplicate	Details in evidence package	
Val-test cross-duplicate	Details in evidence package	
Total groups	13	Varies per group

Stage-D Metal Composition Database

7.11 Metal Composition Table

The Stage-D metal composition database (`metal_composition.db.csv`) contains estimated recoverable metal amounts in grams per device for each supported iPhone model. These values are approximations derived from published environmental reports and academic literature on smartphone material composition.

Table 7.5: Stage-D metal composition database (grams per device).

Model	Gold (g)	Silver (g)	Copper (g)	Aluminium (g)	Cobalt (g)	Titanium (g)	Rare Earths (g)
iphone.8	0.018	0.185	11.75	25	4.25	1.8	0.15
iphone.x	0.021	0.24	13.5	31	5.25	2.0	0.18
iphone.11	0.021	0.22	13.0	35	6.0	2.1	0.20
iphone.13	0.021	0.24	14.75	29	7.5	2.2	0.22

7.12 Fallback Price Table

The fallback CSV (`metal_price_fallback.csv`) provides static USD-per-gram prices for all metals when MetalpriceAPI is unavailable. These values are used as the sole pricing source when the API fails and as supplementary prices for metals without API symbols (titanium, rare earth elements).

Table 7.6: Fallback metal price table (USD per gram).

Metal	Symbol	USD per gram (fallback)	Recovery rate
Gold	XAU	61.50	0.95
Silver	XAG	0.78	0.90
Copper	XCU	0.0094	0.85
Aluminium	ALU	0.0026	0.80
Cobalt	XCO	0.034	0.80
Titanium	N/A	0.011	0.70
Rare Earths	N/A	0.032	0.60

7.13 Metal Value Estimation Formula

The total estimated recyclable value is computed as:

$$V_{\text{BDT}} = \left(\sum_{m \in M} a_m \times r_m \times p_m \right) \times c \quad (7.1)$$

where a_m is the metal amount in grams for metal m , r_m is the recovery rate for metal m , p_m is the price per gram in USD for metal m , and c is the USD-to-BDT

exchange rate. The sum is over all metals M in the device's composition entry.

Training Configuration Details

7.14 Final ViT Model Hyperparameters

Table 7.7: Final ViT training hyperparameter summary.

Parameter	Stage-A (DINOv2)	Stage-B (Swin V2 T)	Stage-C (MaxViT-T)
Learning rate	2e-05	2e-05	2e-05
Batch size	16	16	16
Dropout	0.4	0.4	0.5
Weight decay	0.05	0.05	0.01
Epochs	50	24 (early stop)	100
Early stopping	No	Yes (patience = 15)	No
AMP	Yes	Yes	Yes
Image size	224	224	224
Scheduler	Cosine annealing	Cosine annealing	Cosine annealing
Optimizer	AdamW	AdamW	AdamW
Label smoothing	0.1	0.1	0.1

7.15 Stage-B Anti-Overfitting Experiment Details

The Stage-B anti-overfitting experiment (`vit_stage_b_wd005_fullunfreeze_20260609`) was designed to address a train–val divergence observed in earlier Stage-B runs. The key design choices were:

- **Weight decay $WD = 0.05$** to regularise the backbone weights.
- **Progressive unfreezing:** backbone was frozen for epochs 1–4 and fully unfrozen at epoch 5 with a reduced learning rate of 9×10^{-6} .
- **AMP enabled** (automatic mixed precision) for both speed and implicit regularisation.
- **Early stopping patience = 15:** training halted at epoch 24; best checkpoint was saved at epoch 9.

The best checkpoint (epoch 9) metrics were: `val_acc` = 95.44%, `val_f1` = 95.42%. Test evaluation of this checkpoint yielded: `test_acc` = 92.12%, weighted F1 = 92.08%.

7.16 CNN Baseline Experiments

DenseNet121 was the primary CNN baseline for all three stages. Training used a standard supervised fine-tuning approach with the same image resolution and augmentation as the ViT experiments (224×224 , same augment pipeline).

Table 7.8: CNN baseline training configuration.

Parameter	Configuration
Architecture	DenseNet121 (pretrained on ImageNet)
Optimizer	Adam
Learning rate	1e-04
Batch size	32
Epochs	30
Image size	224×224
Dropout	Default (network architecture dropout)

7.17 YOLO11x-cls Experiments

YOLO11x-cls was trained using Ultralytics’ standard classification training workflow. The model was fine-tuned from the official YOLO11x-cls pretrained weights.

Table 7.9: YOLO11x-cls training configuration.

Parameter	Configuration
Architecture	YOLO11x-cls (Ultralytics)
Pretrained	Yes (YOLO11x-cls default weights)
Learning rate	Auto (Ultralytics default)
Batch size	16
Epochs	50
Image size	224×224

Evidence Package Contents

7.18 Overview

The evidence package accompanying this thesis contains all artefacts needed to reproduce the experimental results.

7.19 Key Output Files

- `vit_result_20260608_032032.json` — Normal phone pipeline output (iphone_11, normal, resell as-is, fallback pricing).
- `vit_result_20260608_032045.json` — Broken minor-damage output (iphone_11, broken, minor_damage, repair then resell, fallback pricing).
- `vit_result_20260608_032740.json` — Broken major-damage output (iphone_11, broken, major_damage, recycle, fallback pricing).

Table 7.10: Evidence package contents.

Item	Contents
database/	Stage-A, B, C image datasets; Stage-D metal composition database
models/final/	Trained checkpoints: DINOv2 ViT-B/14 (Stage-A), Swin V2 Tiny (Stage-B), MaxViT-Tiny (Stage-C)
models/experiments/	All experiment checkpoints for CNN, YOLO, ViT experiments
results/	Metrics CSVs, per-stage confusion matrix PNGs, training history JSONs
outputs/	Final pipeline JSON outputs and HTML summary reports
run_vit_pipeline.py	Final ViT pipeline script
src/	Training scripts for CNN, YOLO, ViT models
api_server.py	FastAPI backend server
ewaste_thesis_website/	React frontend source
requirements.txt	Python package requirements for reproducibility

- `vit_result_20260608_032919.json` — Broken major-damage output (iphone_x, broken, major_damage, recycle, fallback pricing).
- `vit_metalpriceapi_result_20260608.json` — MetalpriceAPI-enabled run confirming successful retrieval of all metal symbols and BDT exchange rate.

7.20 Reproducing Training

To reproduce the final Stage-B Swin V2 Tiny result:

1. Install requirements: `pip install -r requirements.txt`
2. Prepare the dataset in `database/ewaste_thesis_database/stage_b/` with `train/`, `val/`, and `test/` subdirectories.
3. Run the ViT training script with the Stage-B configuration:
`python src/train_vit_stage_b.py --config wd005_fullunfreeze.`
4. The best checkpoint (epoch 9, val_f1 = 95.42%) is saved to:
`models/final/stage_b.swin_v2_tiny_best.pth.`
5. Evaluate: `python src/evaluate_vit_stage_b.py` should reproduce `test_acc = 92.12%`, weighted F1 = 92.08%.

Note: Exact reproduction may differ slightly due to hardware-level non-determinism in CUDA operations. Rerunning with a different random seed may produce a different best-epoch checkpoint number while achieving similar final test metrics.